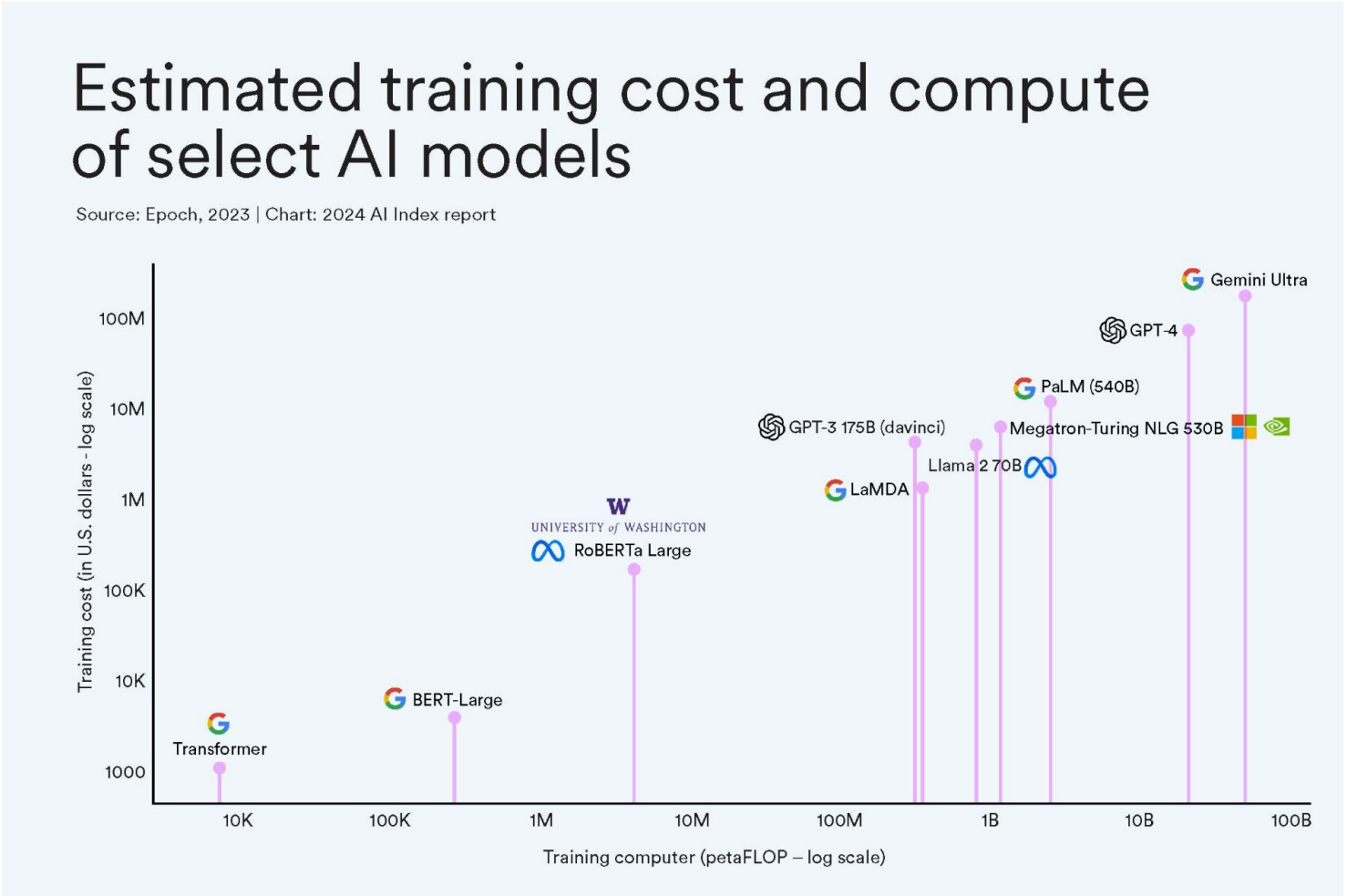
CSE 585

Tanush Mittal, Satyam Goyal, Arjun Laxman, Kushal Patel

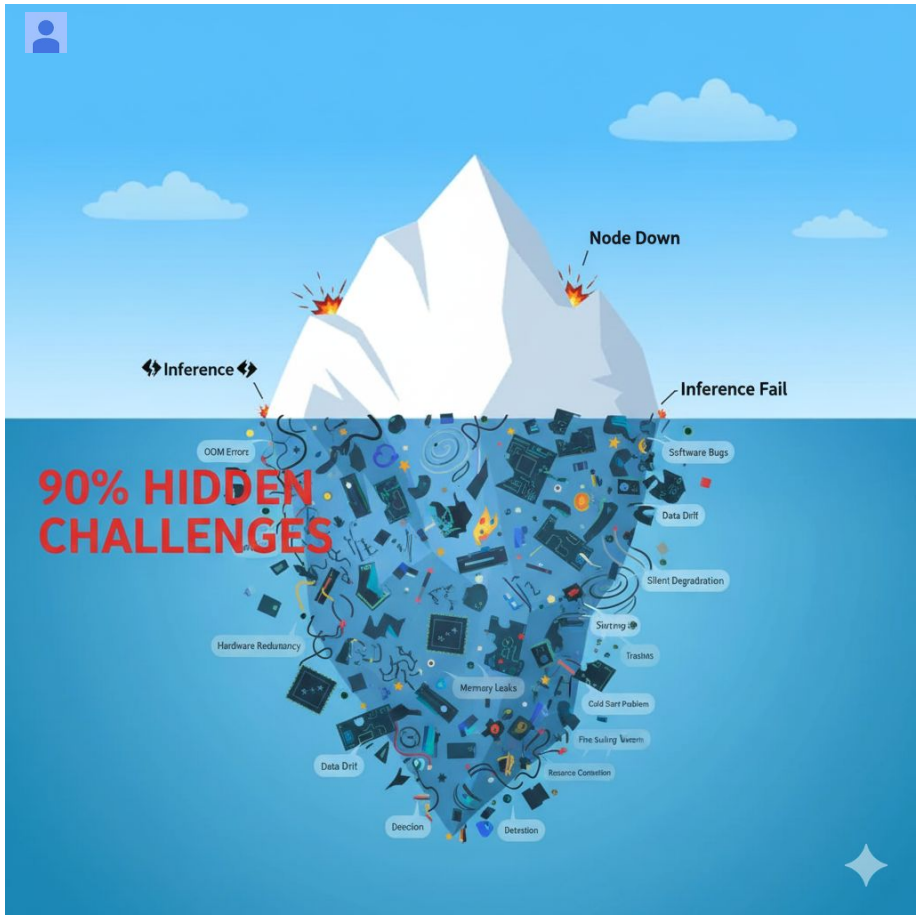
Training LLMs

- More compute
- More GPU Hours
- More $$$$



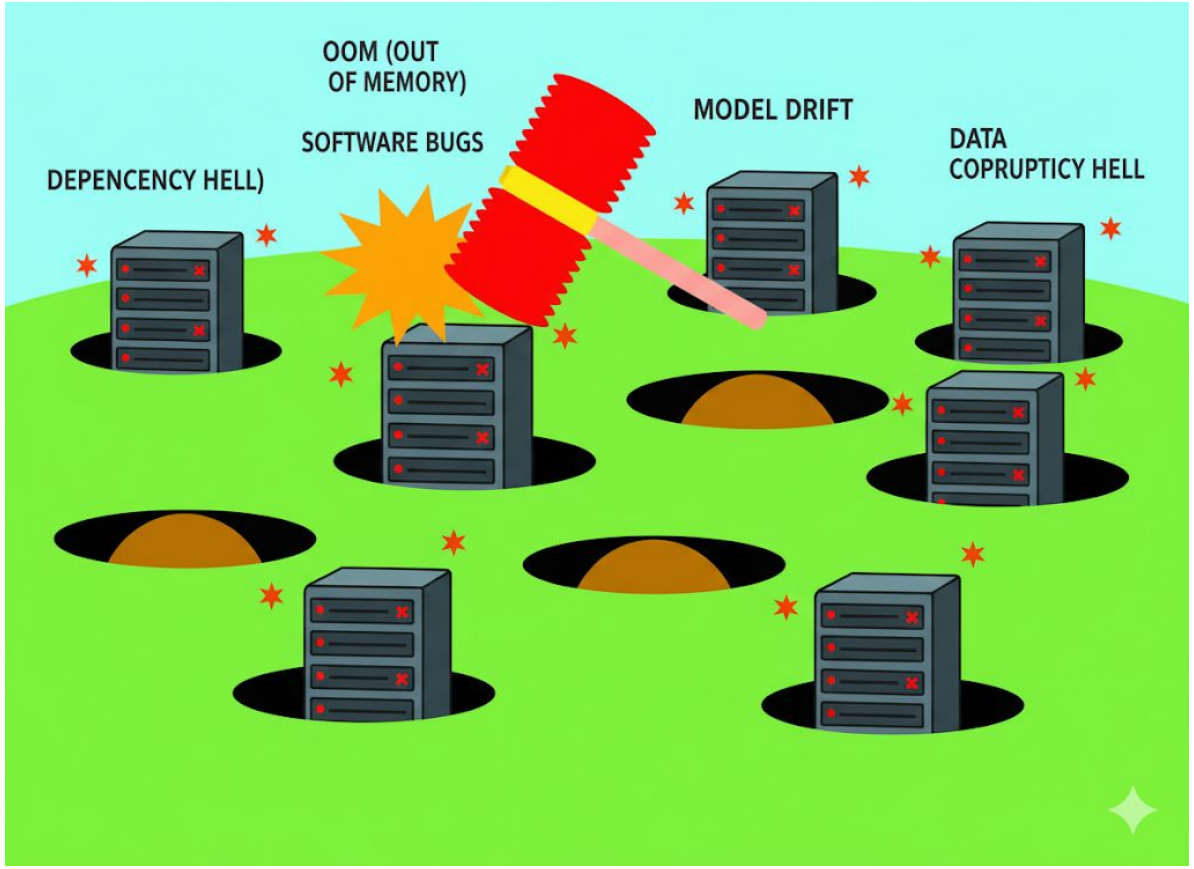
Epoch AI 2023

Training LLMs - reality



90% of problems are hidden

Reactive



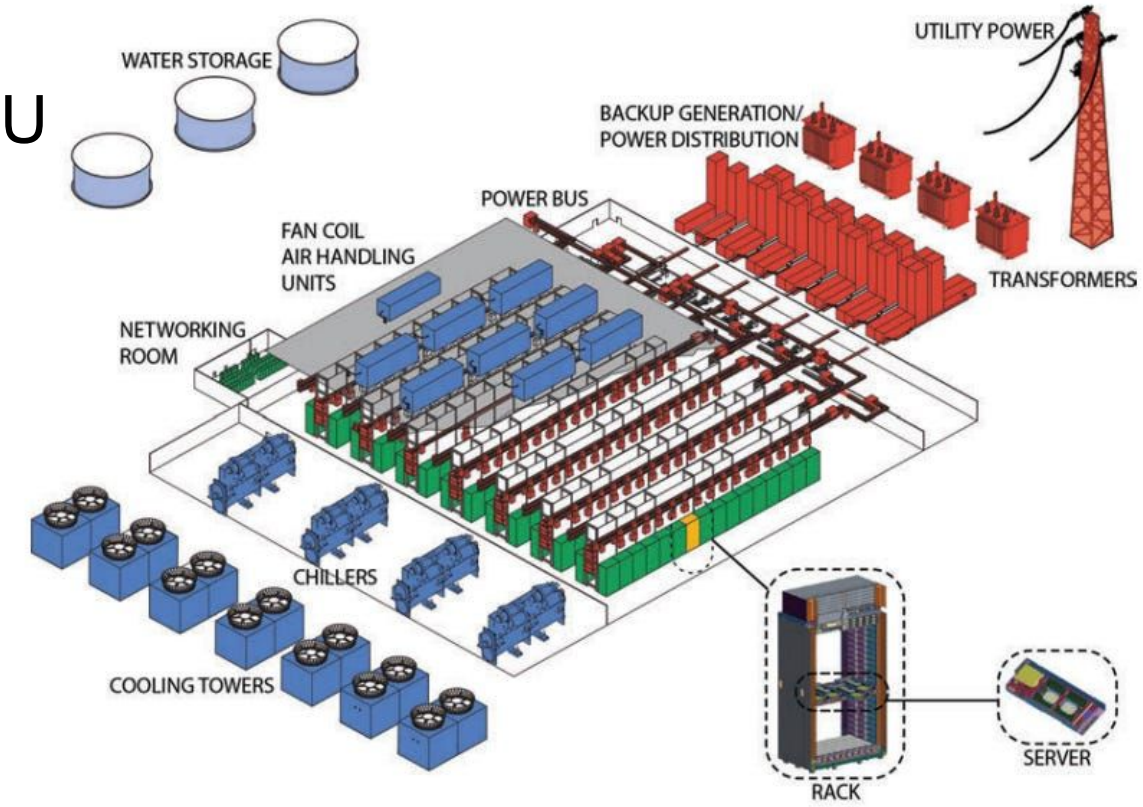
Fix one, miss ten

SuperBench: Improving Cloud AI Infrastructure Reliability with Proactive Validation

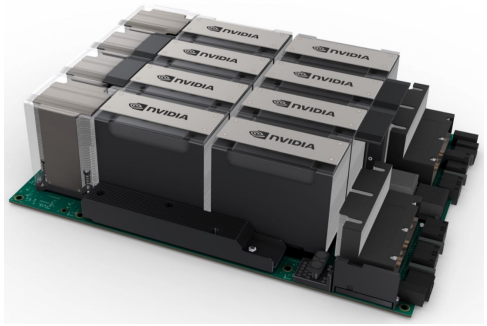
Xiong, Jiang, Yang, Qu, Zhao, Liu, Zhong, Pinzur, Zhang, Wang, Jose, Pourreza, Baxter, Datta, Ram, Melton, Chau, Cheng, Xiong, Zhou

AI infrastructure incidents are costly!

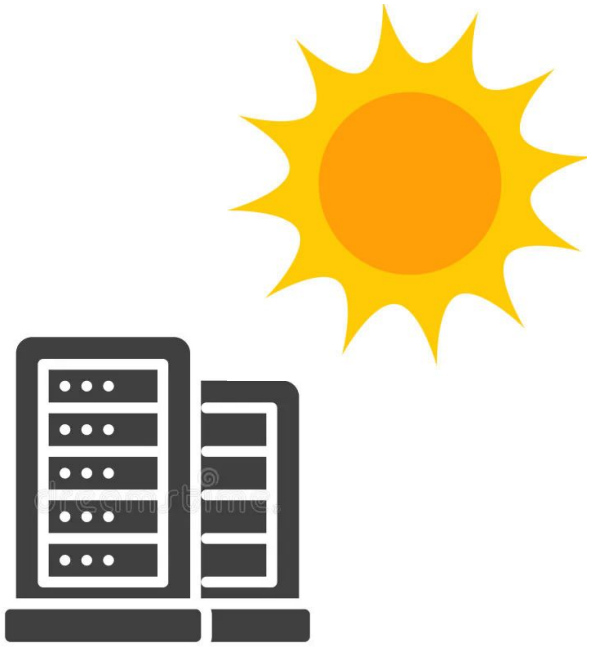
- Meta's OPT training: 105 restarts, 61k GPU hours lost
- Equivalent of running a single GPU 24/7 for nearly 7 years



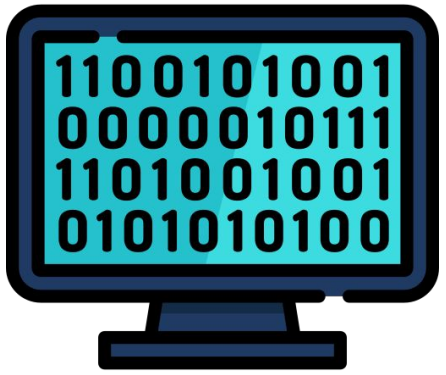
Root Causes of Incidents



Rapid Hardware Evolution



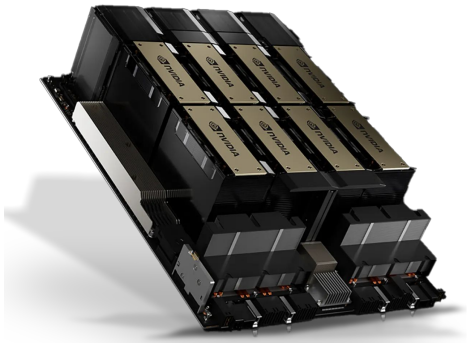
Cloud Environment



Immature Software

Hardware Redundancies

- Hardware redundancies are built into cloud AI infrastructure to increase reliability
- Introduces Gray Failures



GPU Memory Row Remapping

Over-Provisioned Networking Links

Existing Solutions

- Manual troubleshooting
- Reactive approach

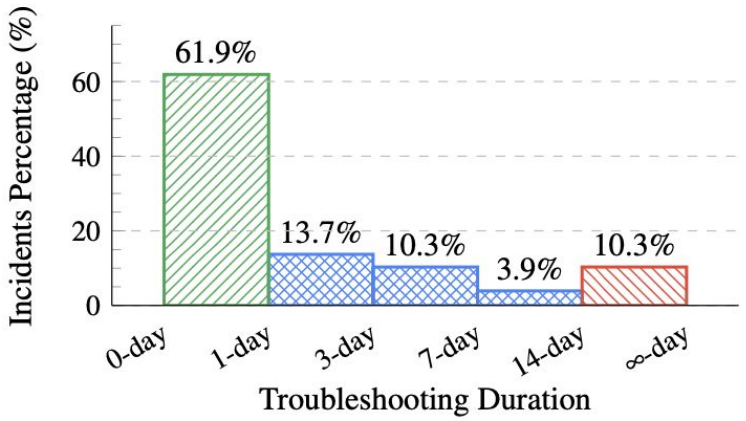
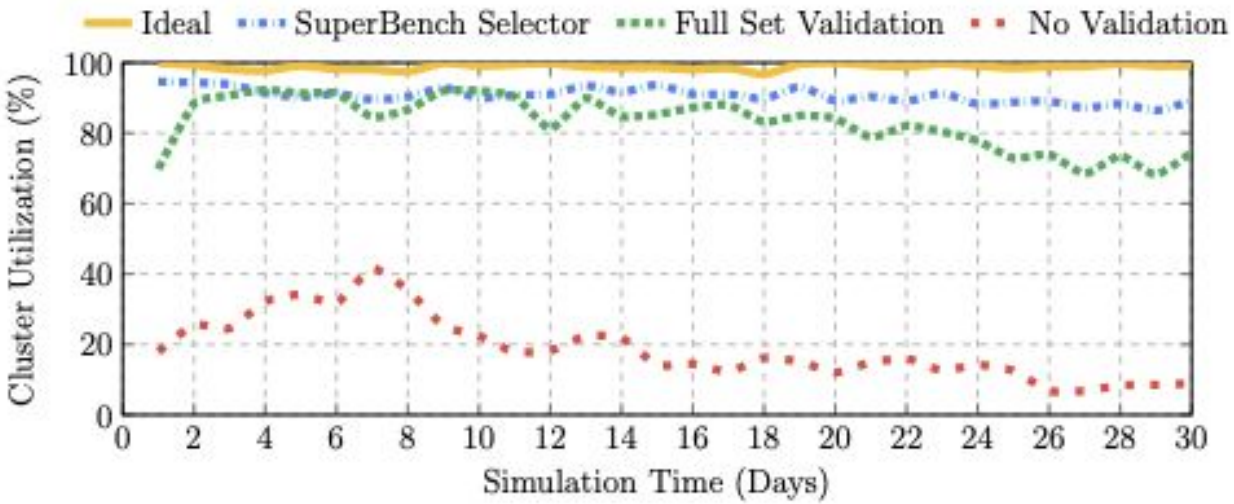


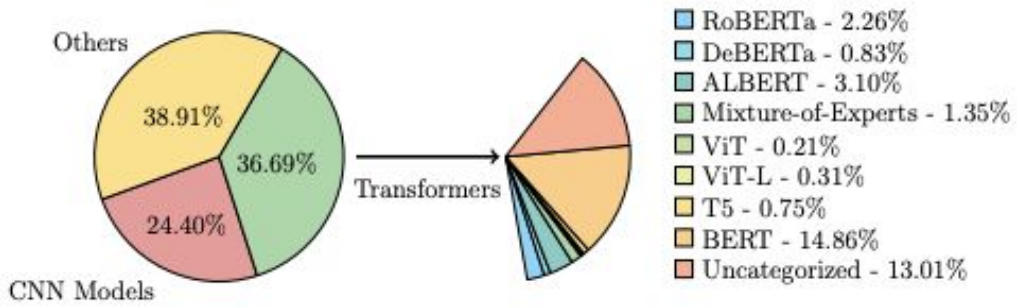
Figure 2: Incidents troubleshooting duration distribution.

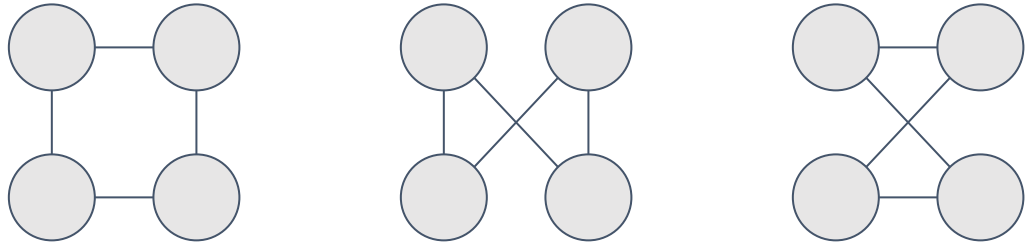
SuperBench's Contribution

- Proactive Approach



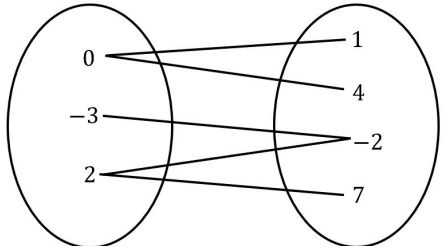
Challenge #1 - Workload Diversity





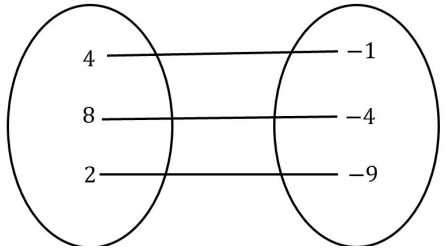
= Nodes = Network Connections

End-to-End Benchmarks



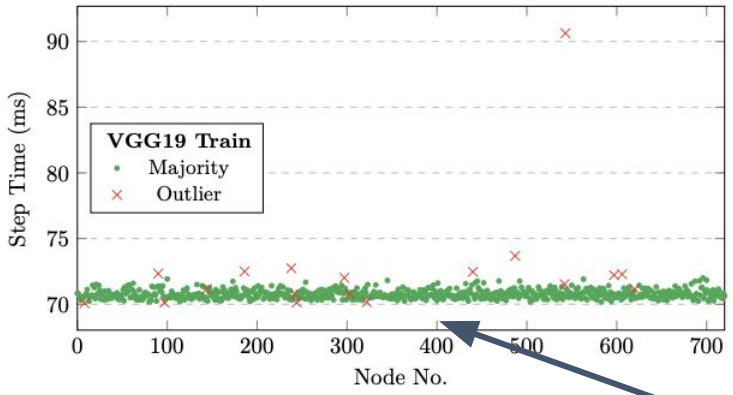
Benchmarks Workloads

Micro-benchmarks



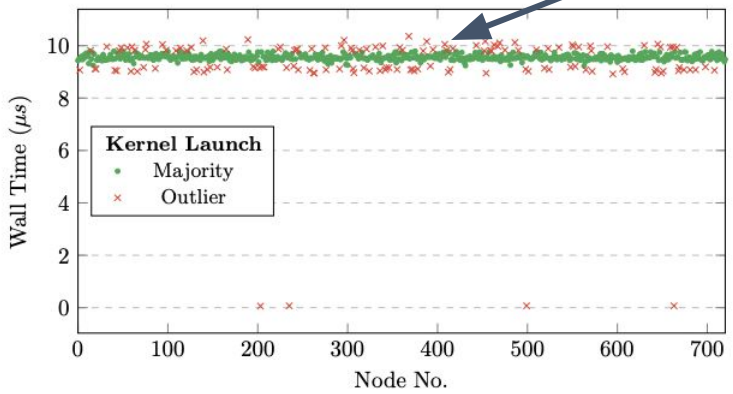
Benchmarks Workloads

Challenge #2 - Lack of Ground Truth

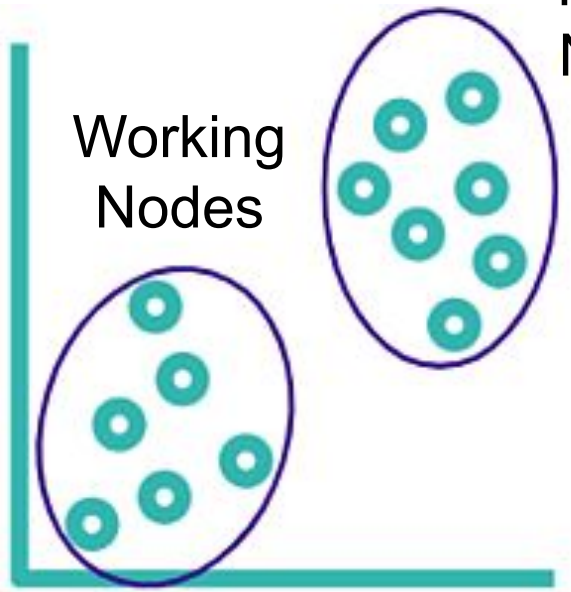


(a) LOF (*neighbors* = 10) on VGG19 training step time

Each benchmark has its own clusters

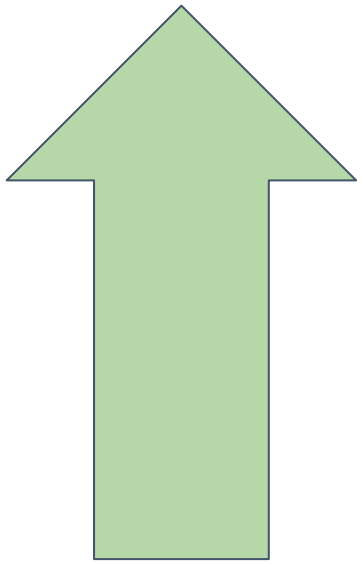


(b) One-Class SVM (ν = 0.2) on GPU kernel launch wall time

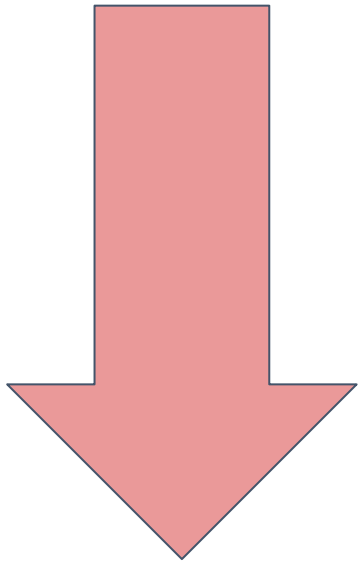


Data Driven Approach

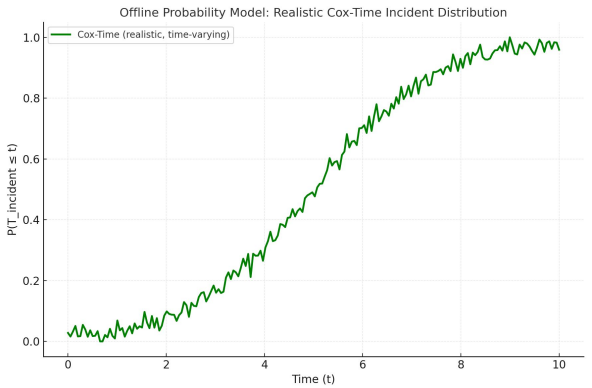
Challenge #3 - Optimal Duration & Coverage



Selected Test Coverage



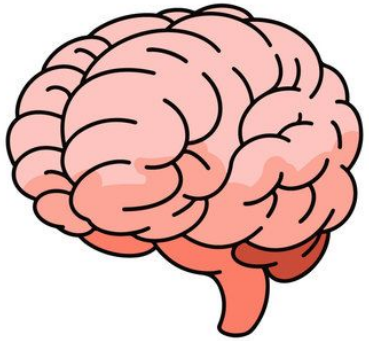
Benchmark Runtime



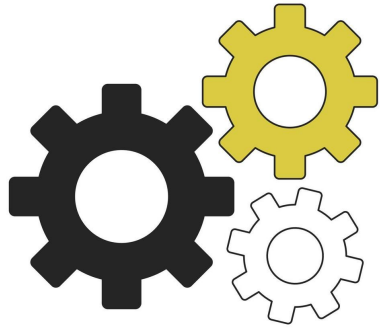
Cox-Time Model

0-1 Knapsack Problem

Core Components



Selector



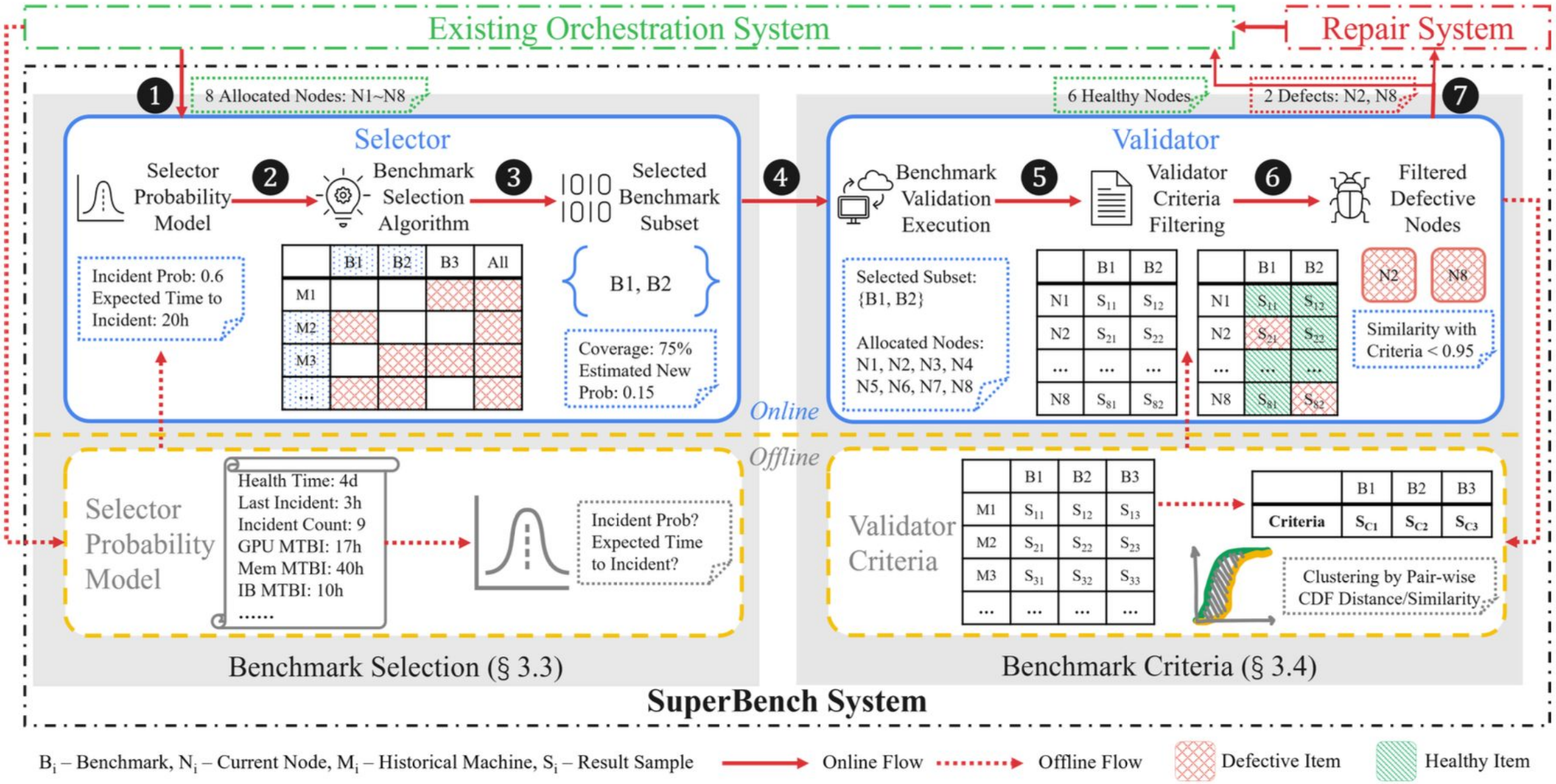
Validator

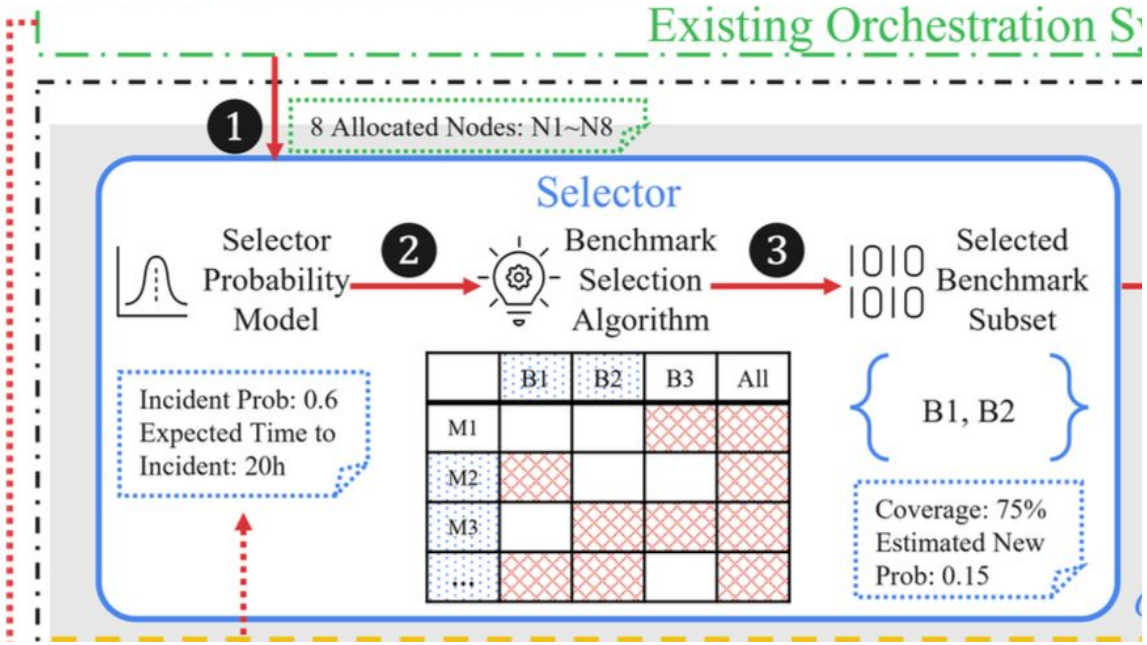
Failure Probability

Benchmark Selection

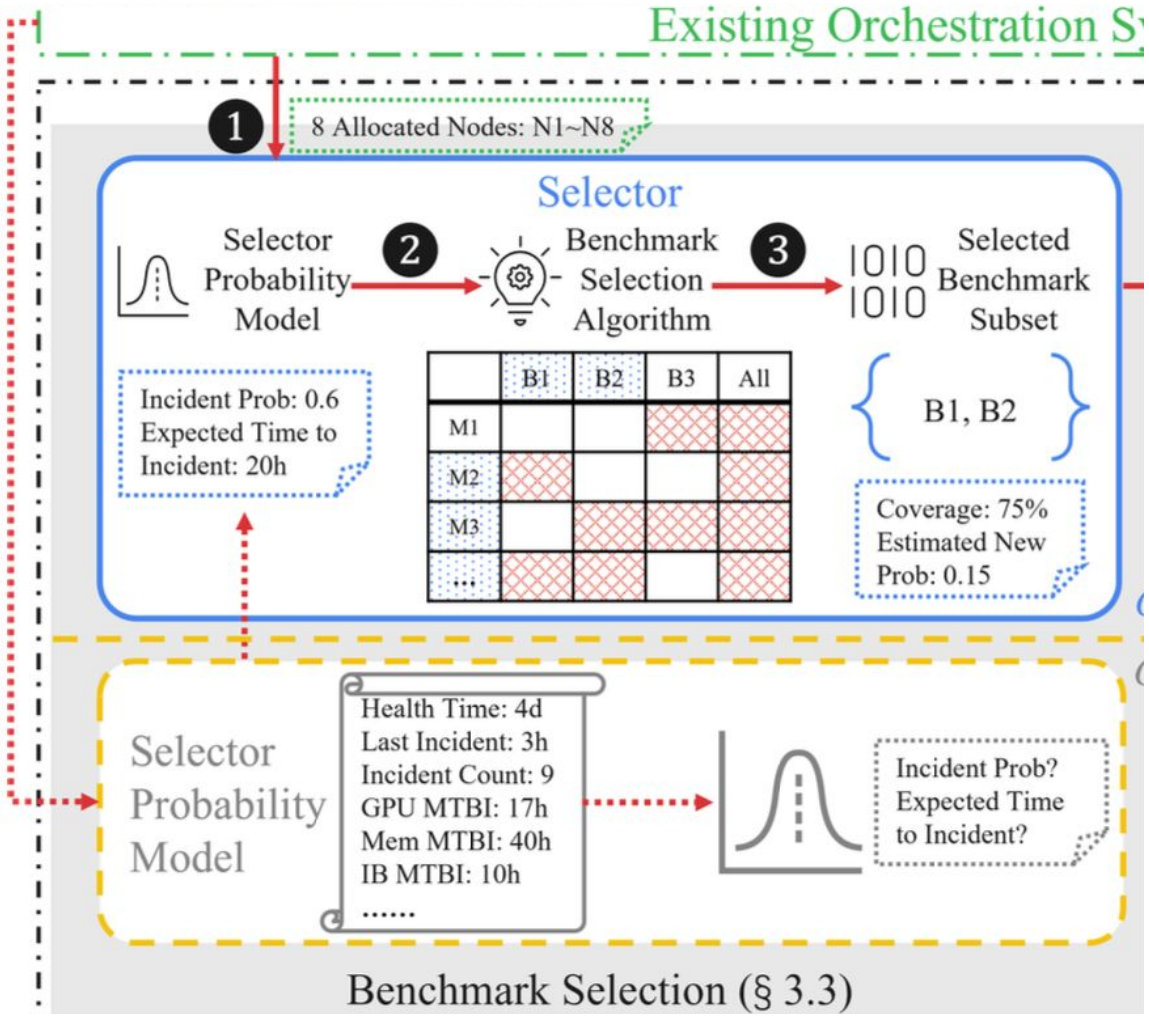
Pass Criteria

Defect Filtering


Existing Orchestration System
Repair System
1
8 Allocated Nodes: N1~N8
6 Healthy Nodes
2 Defects: N2, N8
7
Selector
Selector Probability Model
2
Benchmark Selection Algorithm
3
Selected Benchmark Subset
4
Validator
Benchmark Validation Execution
5
Validator Criteria Filtering
6
Filtered Defective Nodes
Incident Prob: 0.6
Expected Time to Incident: 20h
B1 B2 B3 All
M1
M2
M3
...
B1, B2
Coverage: 75%
Estimated New Prob: 0.15
Selected Subset: {B1, B2}
Allocated Nodes: N1, N2, N3, N4 N5, N6, N7, N8
B1 B2
N1 S11 S12
N2 S21 S22
... ...
N8 S81 S82
N2
N8
Similarity with Criteria < 0.95
Online
Offline
Selector Probability Model
Health Time: 4d
Last Incident: 3h
Incident Count: 9
GPU MTBI: 17h
Mem MTBI: 40h
IB MTBI: 10h
.....
Incident Prob? Expected Time to Incident?
Validator Criteria
B1 B2 B3
M1 S11 S12 S13
M2 S21 S22 S23
M3 S31 S32 S33
... ...
Criteria SC1 SC2 SC3
Clustering by Pair-wise CDF Distance/Similarity
Benchmark Selection (§ 3.3)
Benchmark Criteria (§ 3.4)
SuperBench System
Bi – Benchmark, Ni – Current Node, Mi – Historical Machine, Si – Result Sample
Online Flow
Offline Flow
Defective Item
Healthy Item



1. Event-triggered or does regular validation
2.



1. Event-triggered or does regular validation
2. Failure probability model to predict incident probability for all nodes
3.

Selector: Failure Probability Model

- This model predicts the probability that a node incident will occur by a certain time (t).
- The model uses a Cox-Time model with a neural network to account for factors that change over time, such as gradual performance degradation.
- It is trained on historical data about node statuses, including uptime, incident counts, and mean time between incidents (MTBI).

$$P(T_{incident} \leq t) = \int_0^t f(s)\,ds = F(t)$$

Selector - Offline Probability Model

Goal: Apply Cox-Time Model to determine if a given node is in risk of failing before the end of the next task.

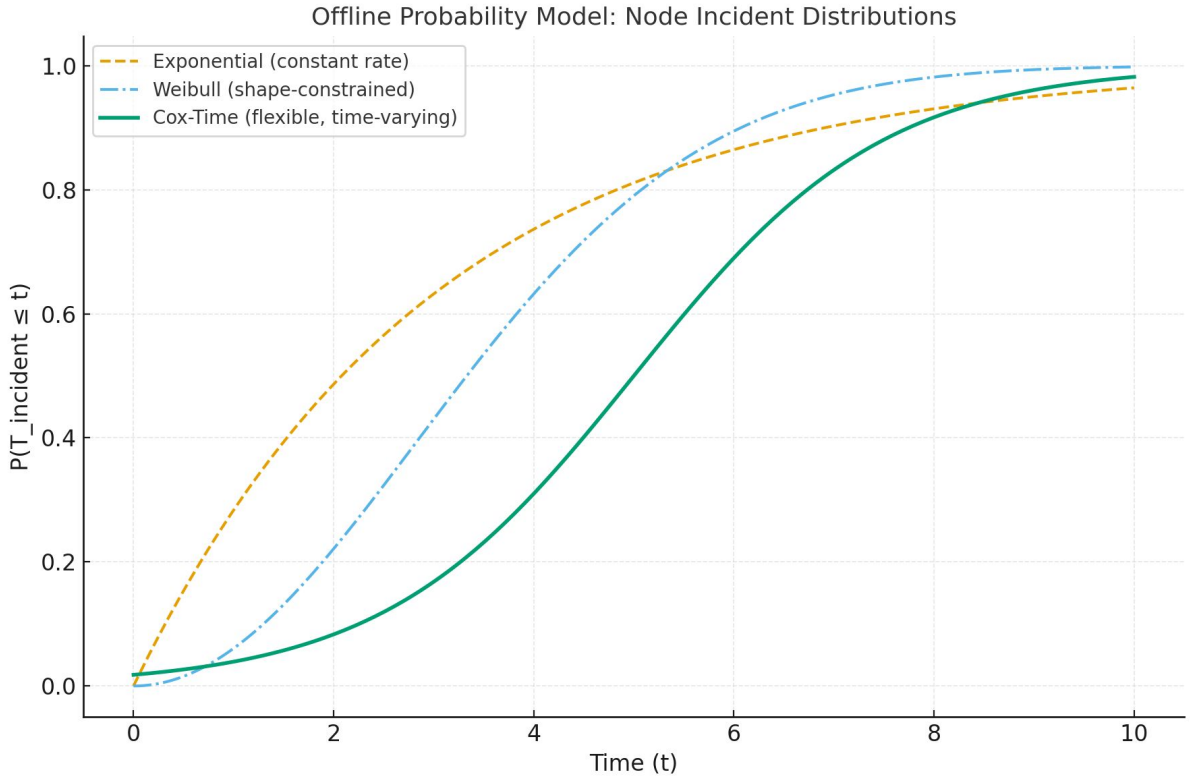
Inputs

-Total up time

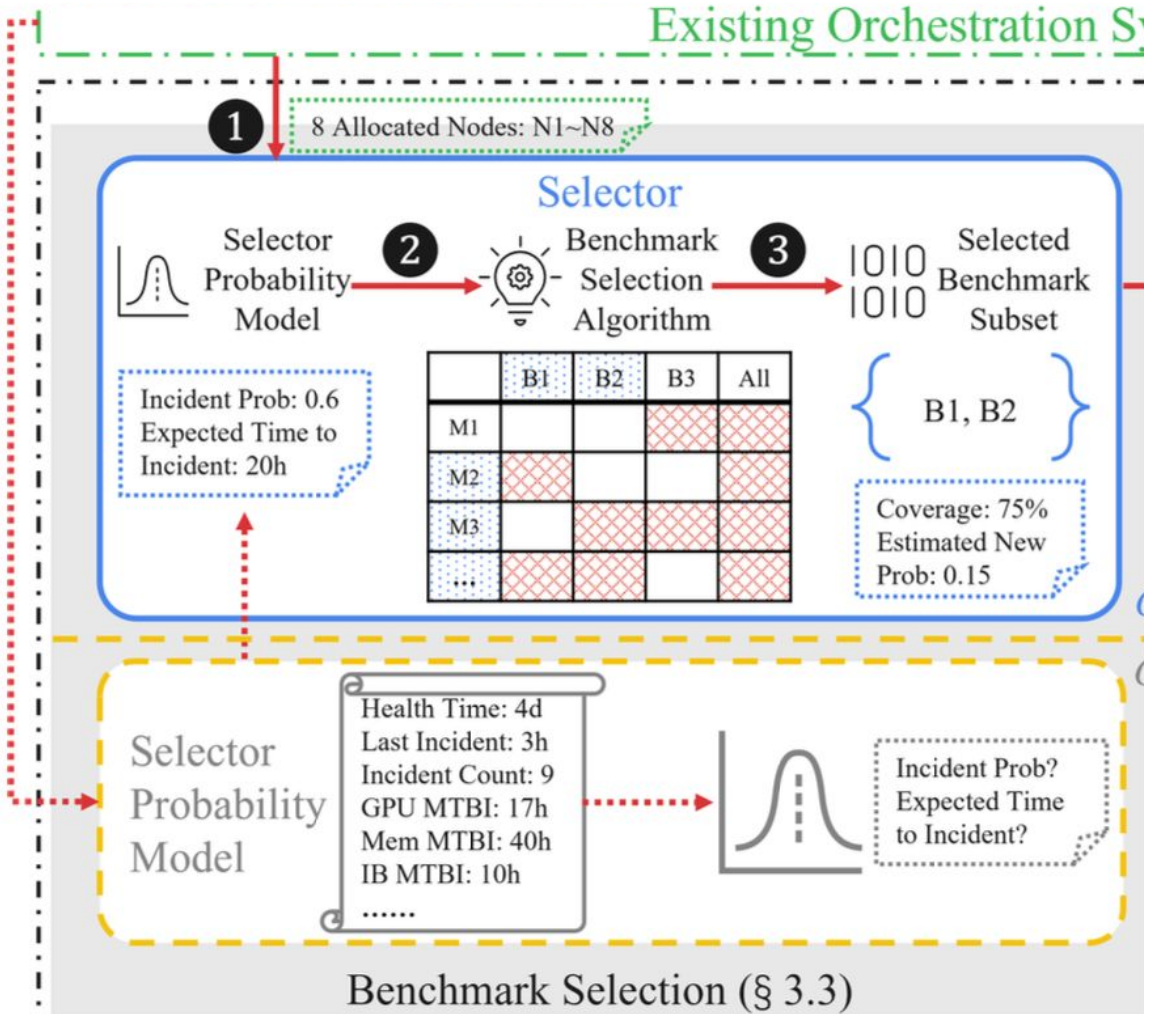
-Time since the last incident

-Historical incident counts

-Mean Time Between Incidents (MTBI)



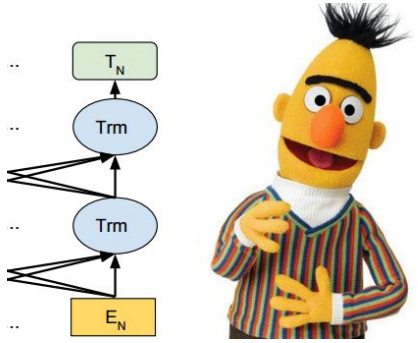
$$P(T_{incident} \leq t) = \int_0^t f(s)\,ds = F(t)$$

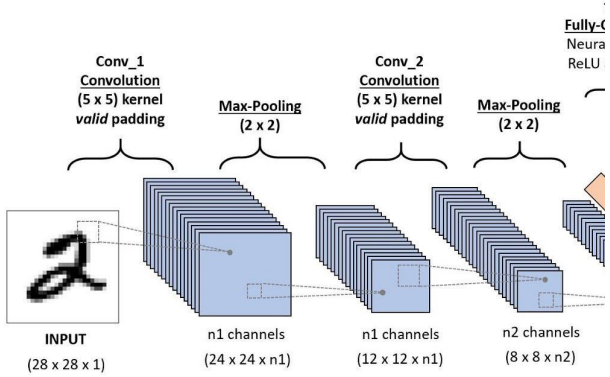


1. Event-triggered or does regular validation
2. Offline Cox-Time model to predict incident probability
3. If probability is too high, pick benchmarks

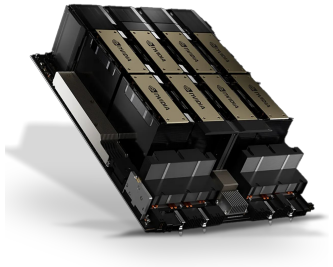
Building a Benchmarking Set

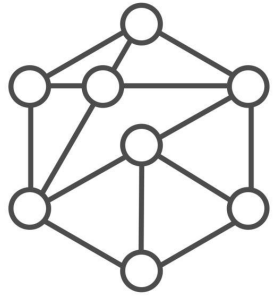
End to End Benchmarks





Micro-benchmarks





Component Wise

Pattern-wise

Building a Benchmarking Set

Table 2: Full benchmark set in SuperBench.

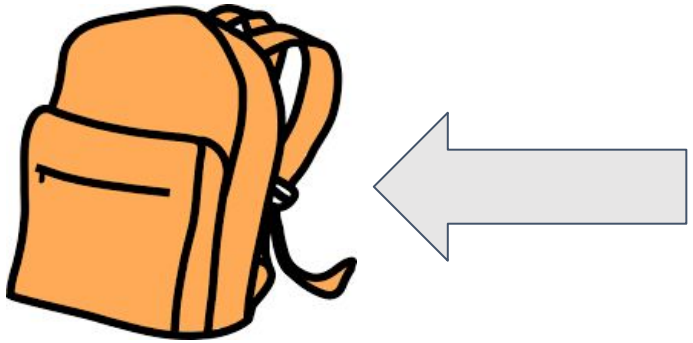
	Micro benchmarks	End-to-end benchmarks
Single Node Phase	Computation • GPU kernel launch • GPU GEMM [5, 45] • cuBLAS kernels • cuDNN kernels • GPU burn Communication • CPU latency [31] • GPU H2D/D2H bandwidth • GPU copy bandwidth • NVLink all-reduce • IB HCA loopback [56] • IB single-node all-reduce Comput./Comm. Overlap • MatMul/all-reduce overlap • Sharding MatMul Disk IO • FIO rand/seq read/write [10]	Multi-GPU training • CNN models – ResNet [27] 50/101/152 – DenseNet [29] 169/201 – VGG [60] 11/13/16/19 • RNN models – LSTM • Transformers – BERT [62] base/large – GPT-2 [15] small/large • Long-running stress – GPT-2 large

Table 2: Full benchmark set in SuperBench.

	Micro benchmarks	End-to-end benchmarks
Multiple Node Phase	Networking • All pair RDMA verbs • GPU collective communication [6, 47] – all-reduce – all-gather – all-to-all	Multi-node training • CNN models • RNN models • Transformers • Long-running stress (same as above)

Selector - Online Selection Algorithm

Goal: Choose an optimal subset of benchmarks that minimizes the total validation time while ensuring sufficient coverage



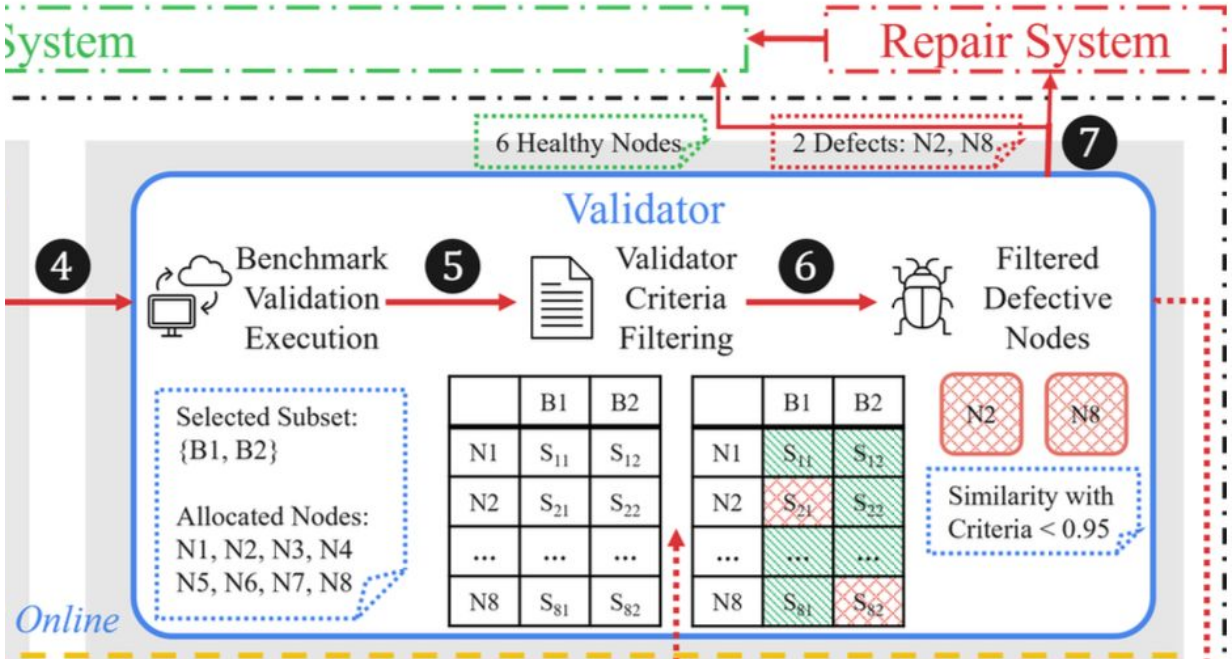
Item: available benchmark

Value: ability to reduce the incident probability

Weight: time

4. Pass benchmark subset to validator

5. Run the benchmarks and get the results

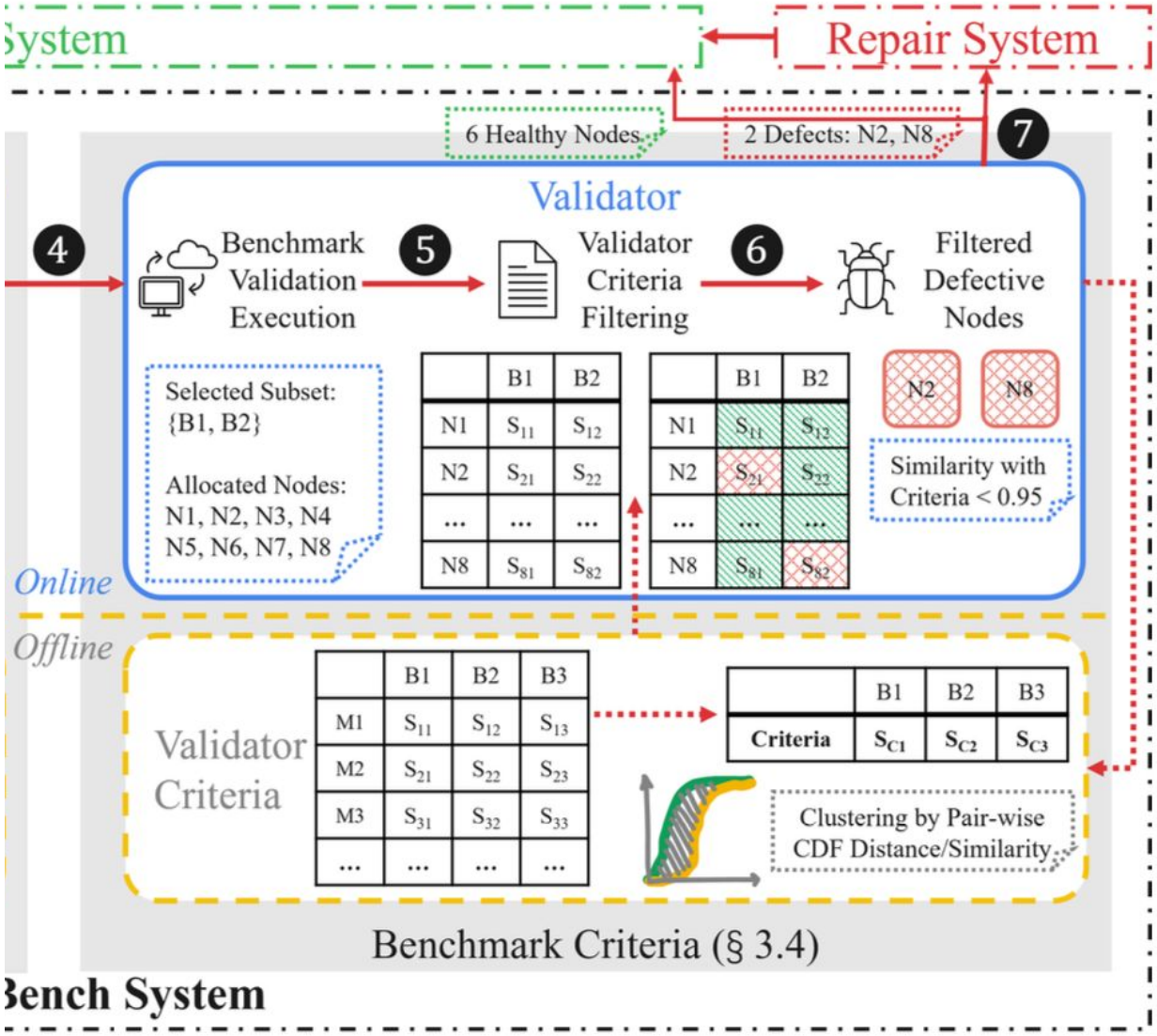


4. Pass benchmark subset to validator

5. Run the benchmarks and get the results

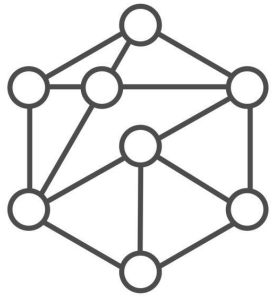
6. Compare offline and current criteria to filter out defective nodes.

7. Output defective nodes

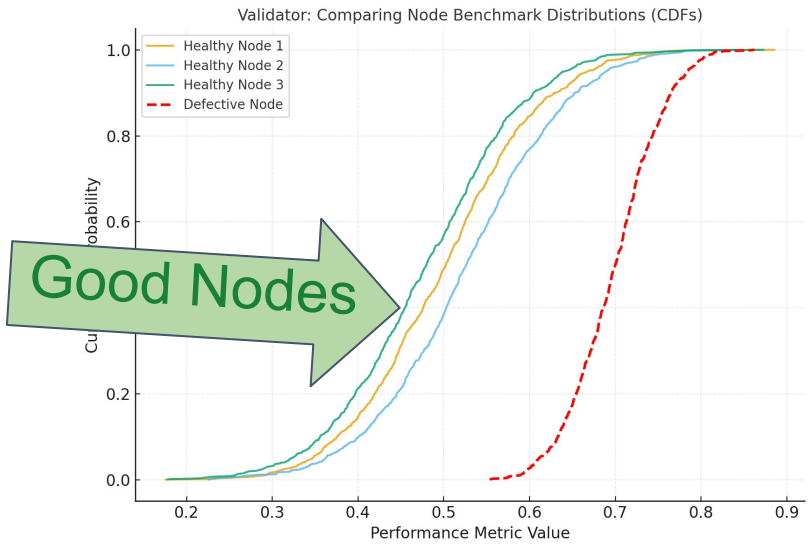


Validator - Offline Criteria Setting

Goal: To establish a clear-cut performance criteria for what constitutes a "healthy" node.

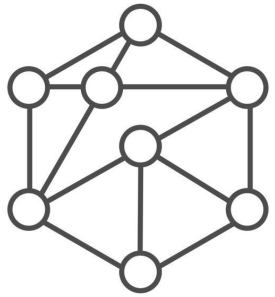


Benchmark Nodes

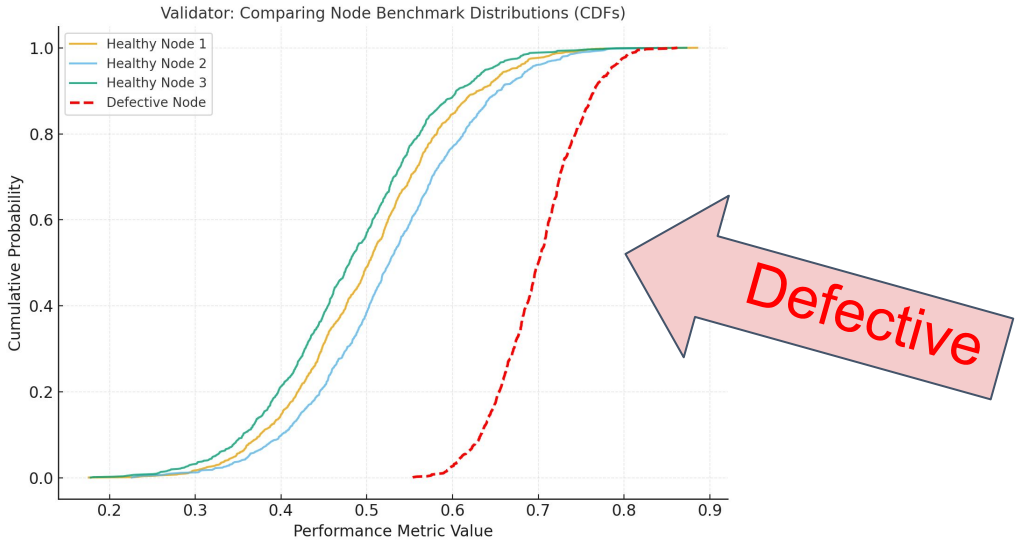


Validator - Online Defects Filtering

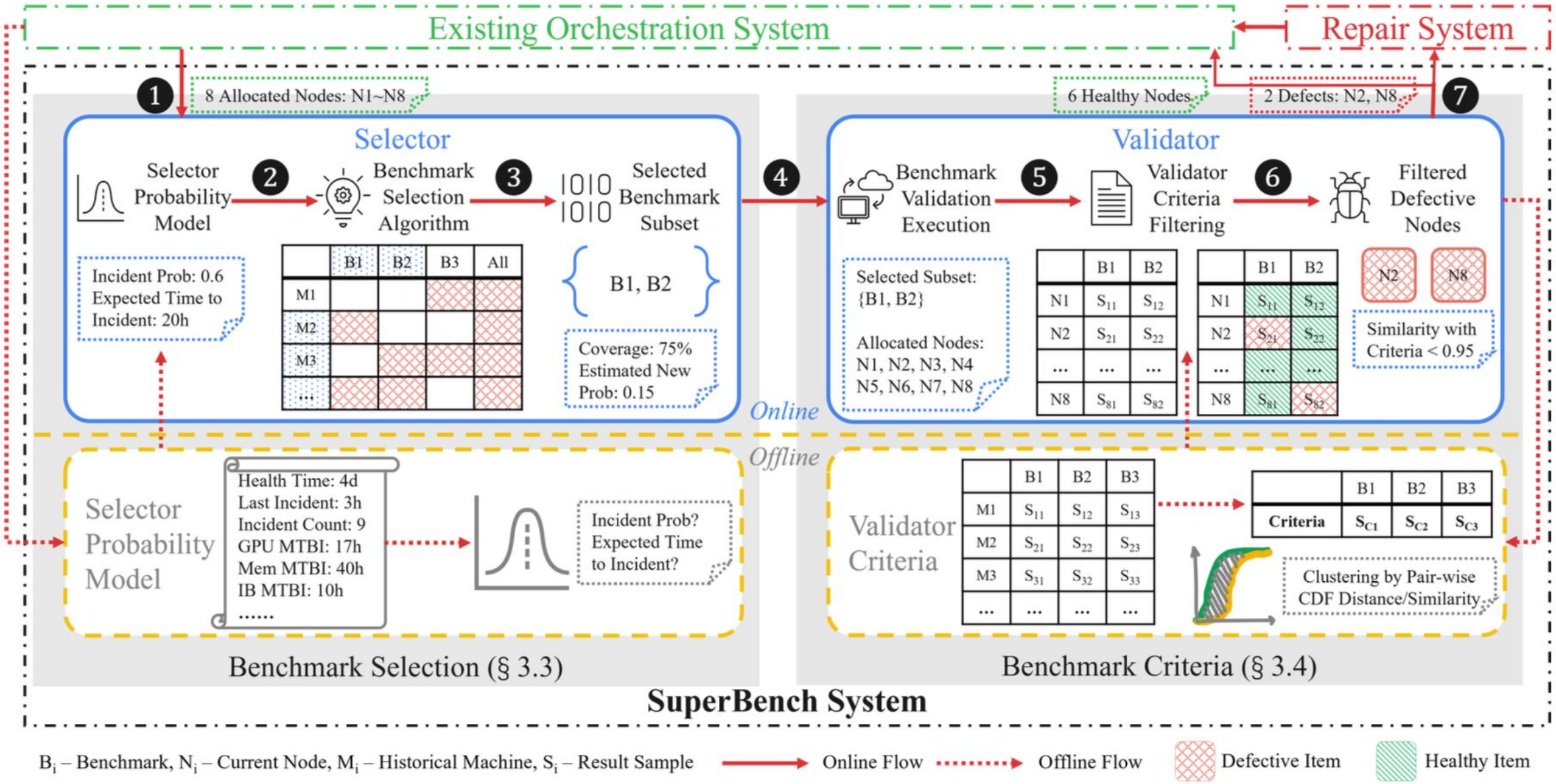
Goal: To identify defective nodes in real-time by comparing their performance against offline-learned criteria.



Benchmark Nodes

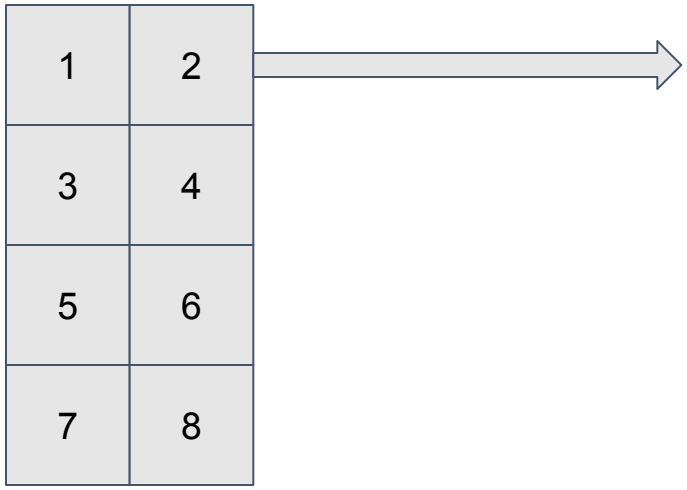


$$d_{1-side}(S_{Obs}, S_C) = \int_0^{\infty} \frac{\max(0, CDF_{S_{Obs}}(x) - CDF_{S_C}(x))}{\max(CDF_{S_{Obs}}(x),\ CDF_{S_C}(x))}\, dx \quad ($$


Existing Orchestration System
Repair System
1
8 Allocated Nodes: N1~N8
6 Healthy Nodes
2 Defects: N2, N8
7
Selector
Selector Probability Model
2
Benchmark Selection Algorithm
3
Selected Benchmark Subset
4
Validator
Benchmark Validation Execution
5
Validator Criteria Filtering
6
Filtered Defective Nodes
Incident Prob: 0.6
Expected Time to Incident: 20h
B1 B2 B3 All
M1
M2
M3
...
B1, B2
Coverage: 75%
Estimated New Prob: 0.15
Selected Subset: {B1, B2}
Allocated Nodes: N1, N2, N3, N4 N5, N6, N7, N8
B1 B2
N1 S11 S12
N2 S21 S22
... ...
N8 S81 S82
N2
N8
Similarity with Criteria < 0.95
Online
Offline
Selector Probability Model
Health Time: 4d
Last Incident: 3h
Incident Count: 9
GPU MTBI: 17h
Mem MTBI: 40h
IB MTBI: 10h
.....
Incident Prob? Expected Time to Incident?
Validator Criteria
B1 B2 B3
M1 S11 S12 S13
M2 S21 S22 S23
M3 S31 S32 S33
... ...
Criteria SC1 SC2 SC3
Clustering by Pair-wise CDF Distance/Similarity
Benchmark Selection (§ 3.3)
Benchmark Criteria (§ 3.4)
SuperBench System
Bi – Benchmark, Ni – Current Node, Mi – Historical Machine, Si – Result Sample
Online Flow
Offline Flow
Defective Item
Healthy Item

Workflow Example

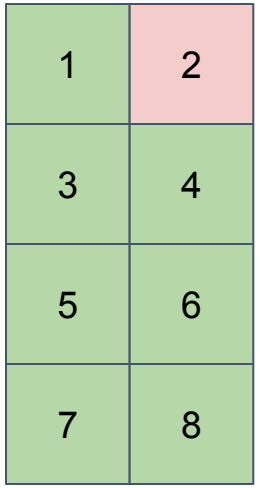
Step 1: Start
Step 2: Offline Cox-Time model to predict incident probability



Node 2

- Uptime: 1 Month
- Last Incident: 1 Week
- Historical Incidents: 3 Incidents
- MTBI: Lower than cluster avg.

= Node



Node 2 has 0.7 chance of incident

Workflow Example

Step 3: If probability is too high, pick benchmarks until < threshold (q = 0.25)
Step 4: Pass benchmark subset to validator

2

p = 0.7

Available Benchmarks

Test	Value (p)	Time (min)	Score (p/min)
Memory	0.35	2	0.175
GPU	0.55	2	0.275
Network	0.15	1	0.15
CUDA	0.24	4	0.06

Iteration 1

- Select GPU
- New Risk = 0.7 * (1 - 0.55) = 0.315

Iteration 2

- Select Memory
- New Risk = 0.315 * (1 - 0.35) = 0.20

Workflow Example

Step 5: Run the benchmarks and get the results
Step 6: Compare offline criteria and current result to filter out defective nodes.
Step 7: Output defective nodes

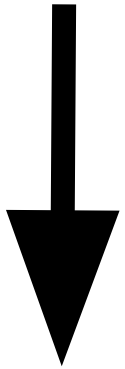
Memory Benchmark
- 99% Similarity

GPU Benchmark
- 30% Similarity

Node 2 is defective

Experiments

- 3k+ A100 VMs (24k+ A100 GPUs total)
- 90 days during cluster build-out phase
  - 24 different benchmarks per VM
  - 2,441 metrics collected per VM



Cluster Benchmark Dataset

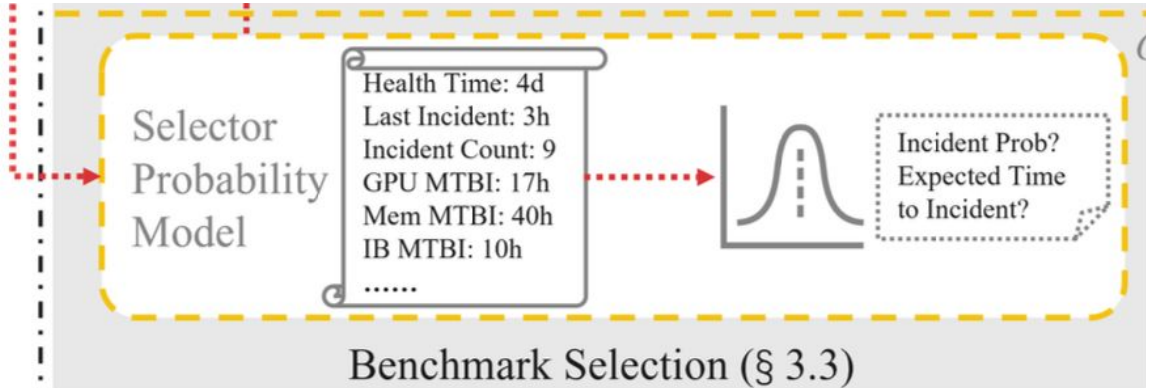
Results - Offline Probability Model

Cox-Time Model Performance:

- Measured as accuracy of time before next incident (TBNI)
- Used 20% of trace data for evaluations

Table 3: Accuracy of different probability models.

Model	Accuracy
Exponential Distribution	75.12%
Exponential Distribution per Incident Count	63.03%
Exponential Distribution per Hour	75.12%
Cox-Time Model	**93.13%**



Results - Online Selection Simulation

30 Day sim

Node time is split between:

- Running customer jobs (what counts toward "utilization")
- Running validation benchmarks (overhead)
- Being down for incidents/repair (downtime)

When jobs fail, nodes go down for 1.5 days troubleshooting

Results - Online Selection Simulation

- Achieved **90.70%** cluster utilization (4.81× improvement over no validation)

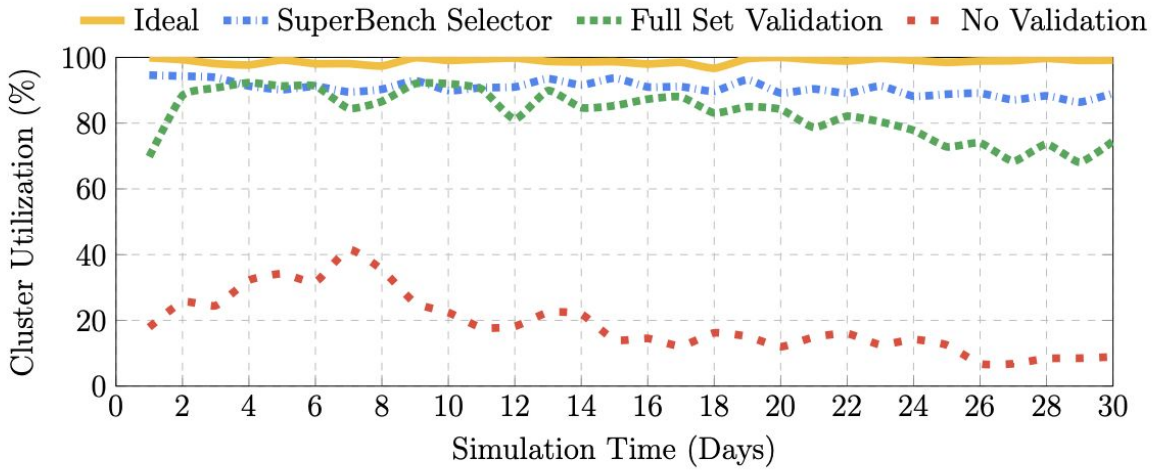


Figure 8: Simulated average node utilization with different benchmark selection policies within 30 days.

Results - Online Selection Simulation

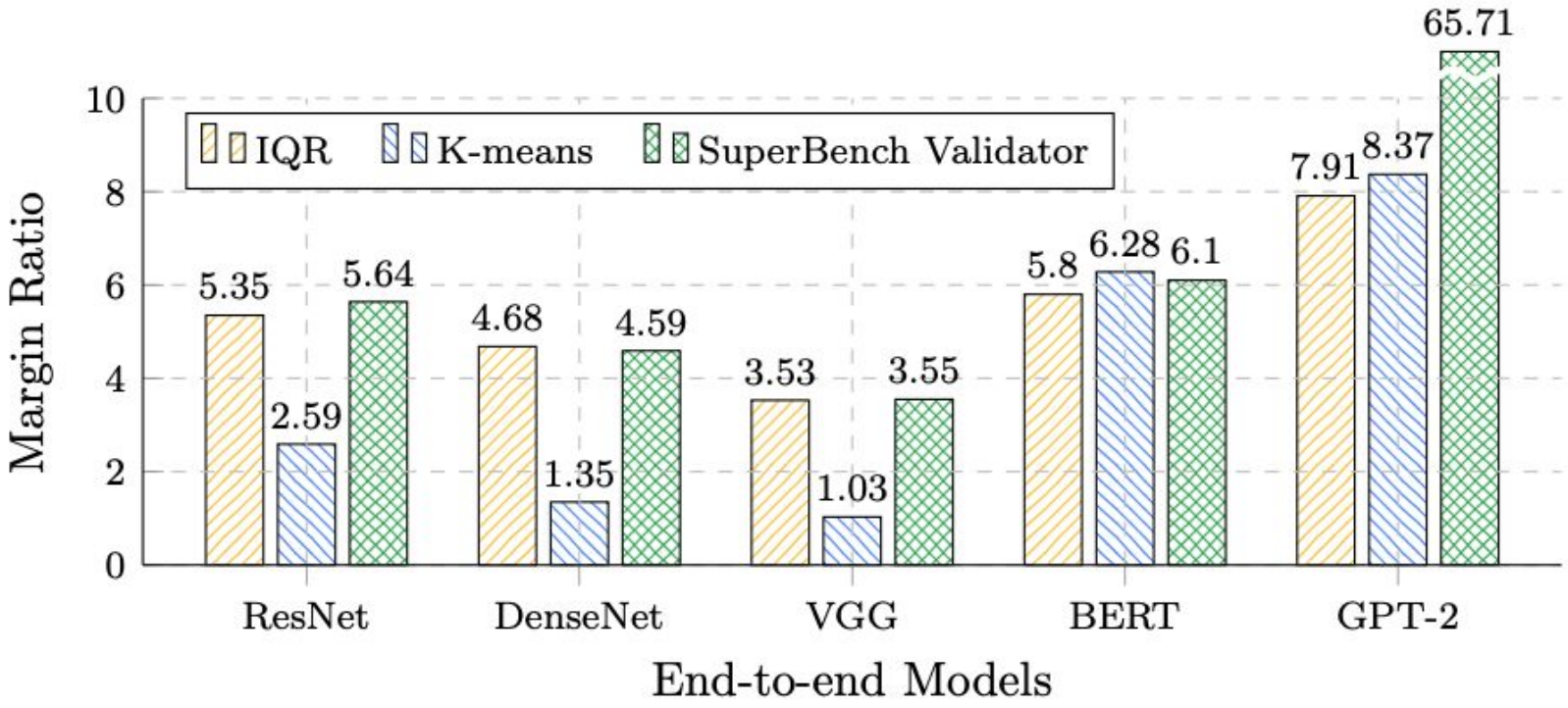
Table 4: Simulated average node validation time and MTBI with different benchmark selection policies in 30-day.

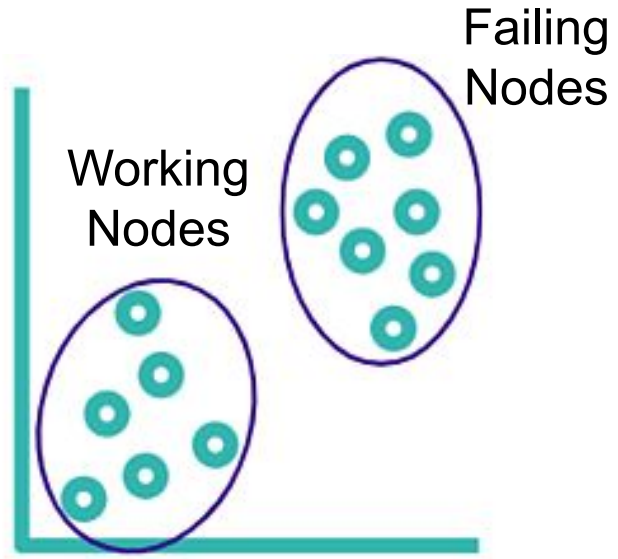
Policy	Absence	Full Set	SuperBench Selector
Validation Time (h)	0	100.40	**7.96**
MTBI (h)	11.59	236.26	**262.05**

- **Increased mean time between incidents (MTBI) by 22.61** × compared to no validation
- Reduced validation time by **92.07%** compared to full set

Results - Criteria and Defects Filtering

- Outperformed IQR and K-means clustering in 4/5 models
- Margin Ratio = min defective distance / max healthy distance to criteria





Results - Benchmark Effectiveness

- Most effective benchmarks: InfiniBand HCA loopback (6.04% defect detection), memory bandwidth tests (2.03%), and BERT models (1.59%)

- Maintained >97% repeatability across all effective benchmarks

Table 5: Repeatability after benchmark parameters tuned.

End-to-end Models	Repeatability (FP32 / FP16) Fixed Parameters	Tuned Parameters	Time Saving FP32 / FP16
ResNet	98.70% / 97.52%	98.70% / 97.55%	73.96% / 78.30%
DenseNet	99.13% / 98.82%	99.12% / 98.76%	73.96% / 73.96%
VGG	98.63% / 97.38%	98.62% / 97.51%	75.59% / 70.70%
LSTM	99.51% / 98.11%	99.52% / 98.11%	73.96% / 73.96%
BERT	99.62% / 99.44%	99.62% / 99.44%	73.96% / 77.21%
GPT-2	99.51% / 99.37%	99.48% / 99.36%	73.96% / 67.45%

Table 6: Effectiveness and repeatability in real deployment.

Benchmark	Repeatability	# Defects / # Total
IB HCA loopback	99.96%	6.04%
H2D/D2H bandwidth	99.68%	2.03%
BERT models	99.39%	1.59%
CPU latency	99.60%	1.33%
IB single-node all-reduce	99.90%	1.10%
ResNet models	99.21%	0.73%
GPT-2 models	99.19%	0.53%
LSTM models	98.66%	0.46%
DenseNet models	97.70%	0.40%
MatMul/all-reduce overlap	97.88%	0.33%
NVLink all-reduce	99.89%	0.30%
GPU GEMM	99.44%	0.23%

Results - Production Impact

- Deployed in Azure for 2+ years across 24k+ A100 GPUs
- Identified 10.36% of nodes as defective during cluster build-out

Key Novelty Aspects

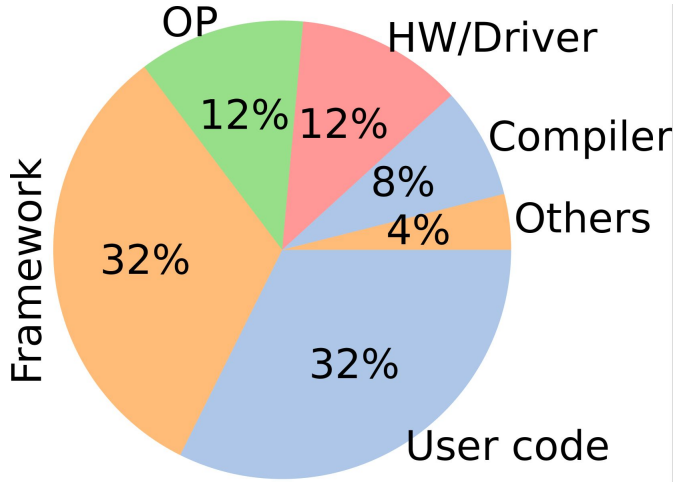
- First systematic analysis of how redundancies cause rather than prevent reliability issues in AI infrastructure
- Application of survival analysis (Cox-Time models) to infrastructure reliability prediction
- Intelligent benchmark selection that dynamically optimizes validation coverage vs. cost

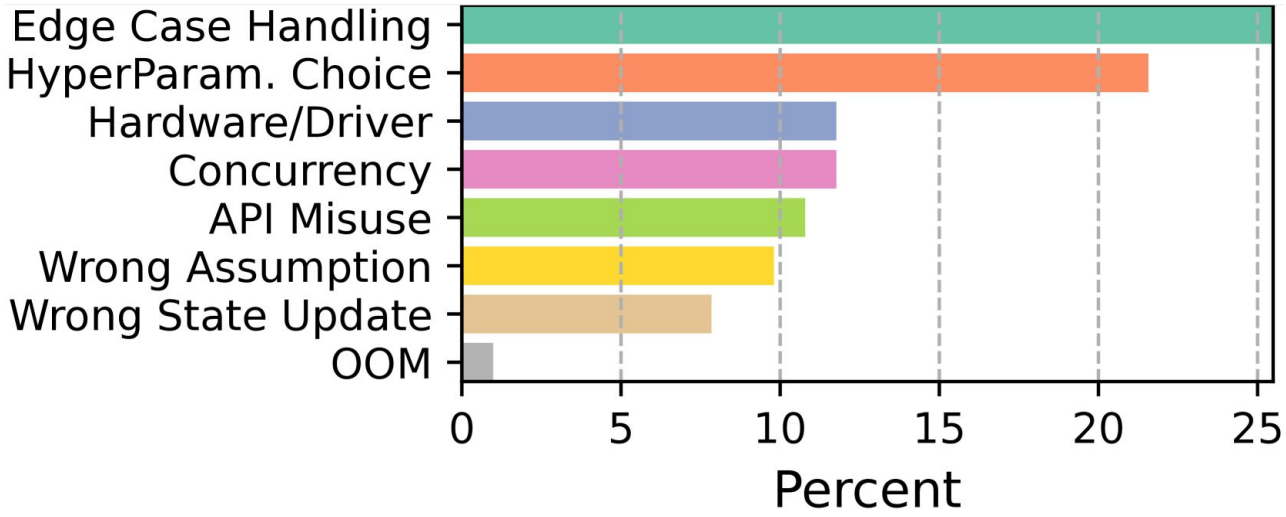
Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks

Yuxuan Jiang, Ziming Zhou, Boyu Xu, Beijie Liu, Runhui Xu, and Peng Huang

Silent Errors While Training

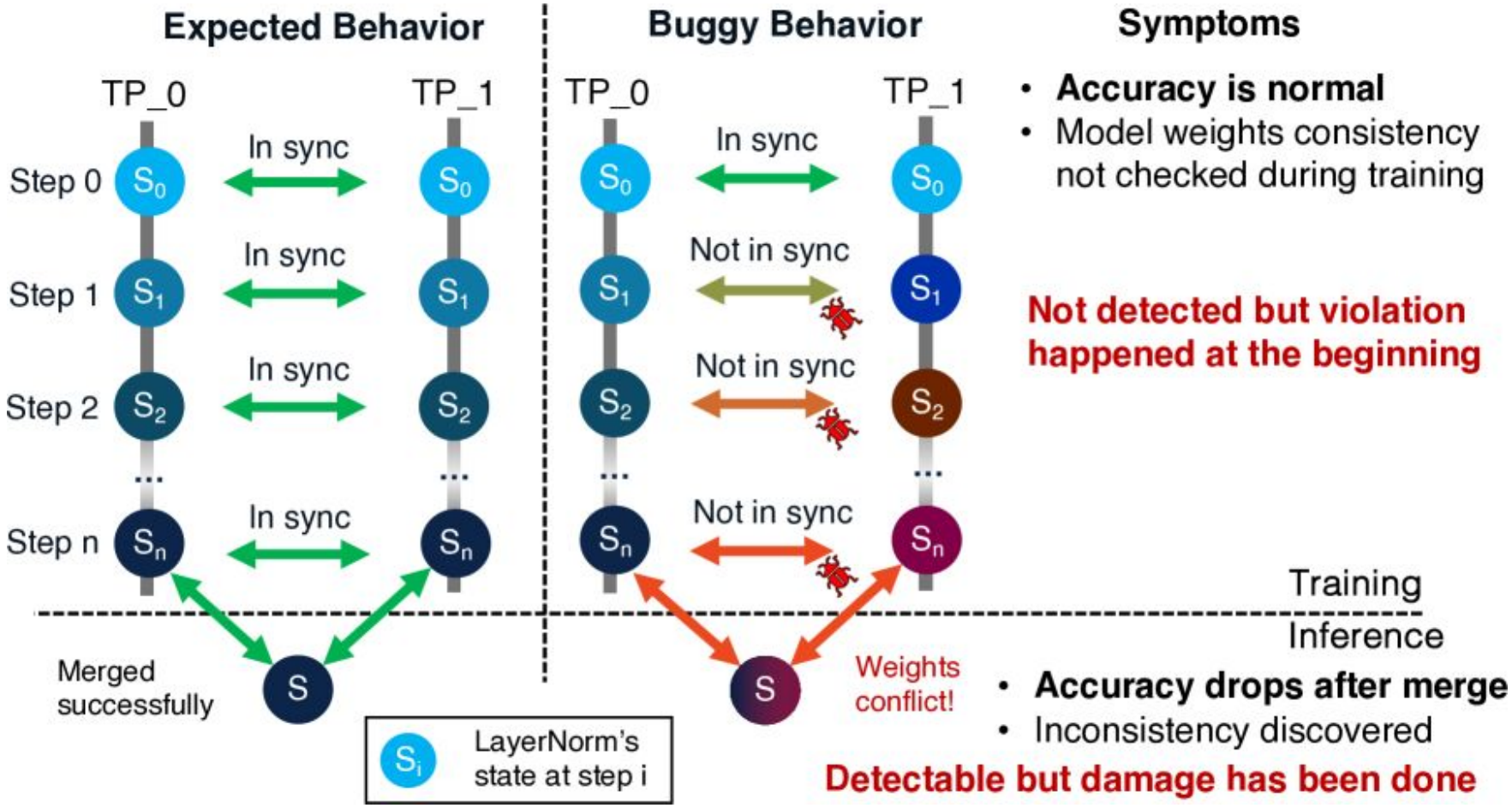
- Training Deep Learning(DL) models is **incredibly complex** leading to **silent errors**: errors that are not obvious but cause **performance/accuracy issues**





Example: Training BLOOM-176B

- Weight divergence when Hugging Face was training BLOOM-176B



- Led to higher loss, perplexity in model

- Required 10 days to detect and 9 days to merge weight/mitigate impact

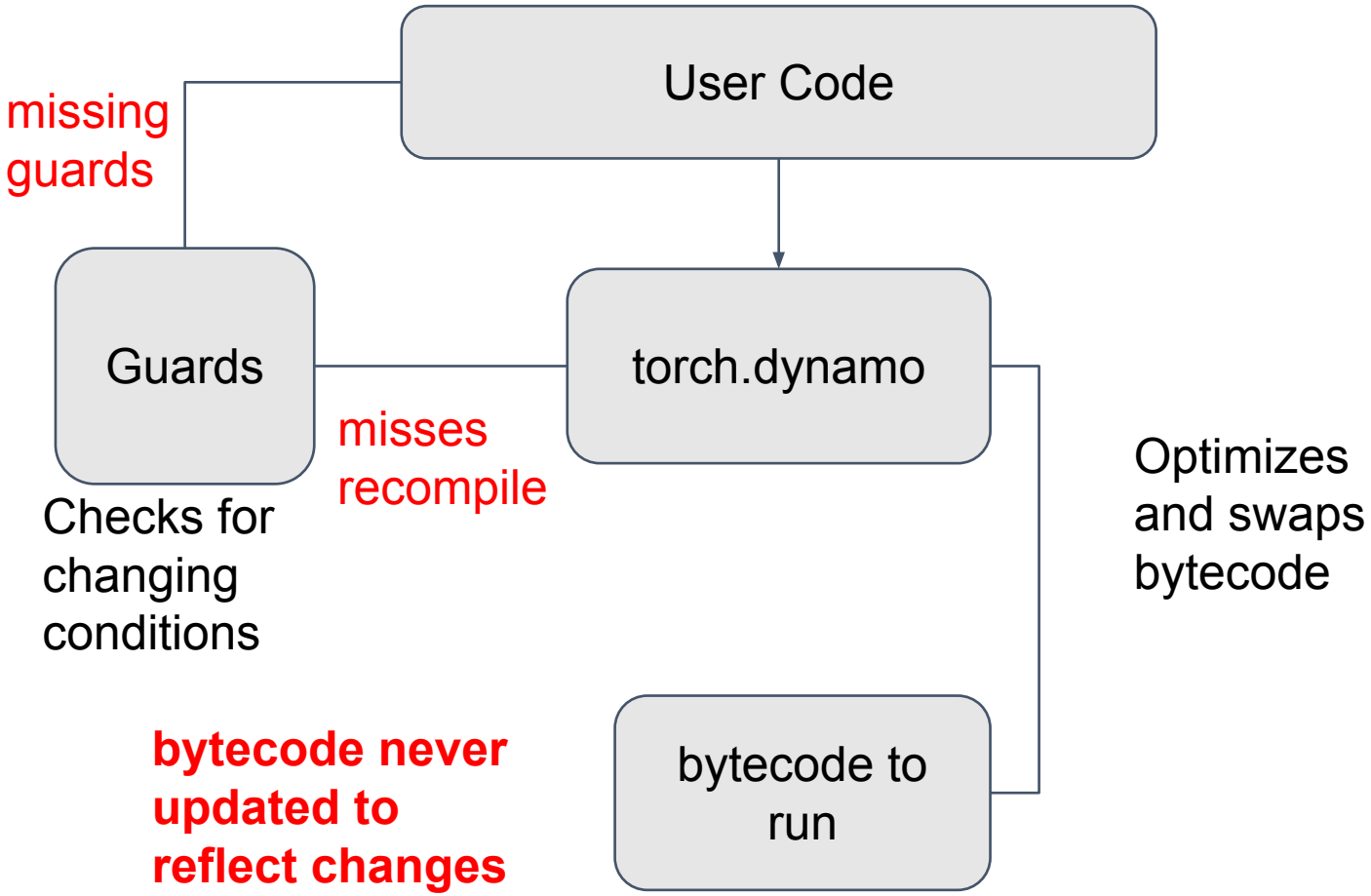
- **Significant loss in training resources and time**

Iter	Type	Loss Diff	PPL Diff	Diff (Loss/PPL)
2000	Valid	+1.14%	+1.43%	+0.014 / +0.050
2000	Test	+2.74%	+3.30%	+0.032 / +0.107
4000	Valid	+3.05%	+3.36%	+0.033 / +0.099
4000	Test	+4.67%	+4.79%	+0.047 / +0.131

Reproducing DeepSpeed-1801 (the root cause of BLOOM176B silent training error) in a small transformer-based language model.

Example: PyTorch 115607

Behavior



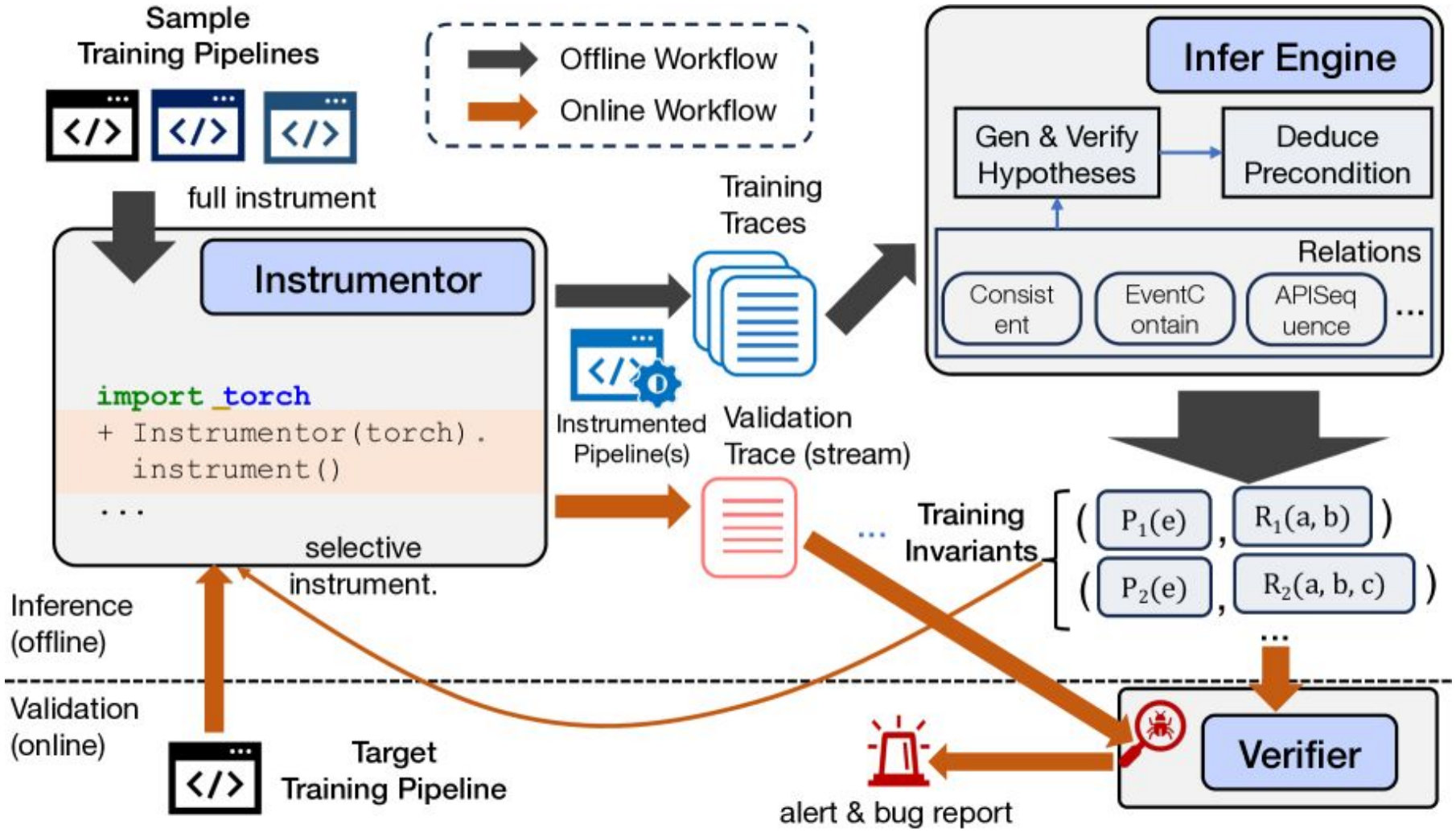
Why is this such an Issue?

- Difficult to know if there is an error
  - Is the model **taking time to converge** or is there an error?
- Hard to dissect with high level metrics such as loss and accuracy
- **Huge waste of time and resources**

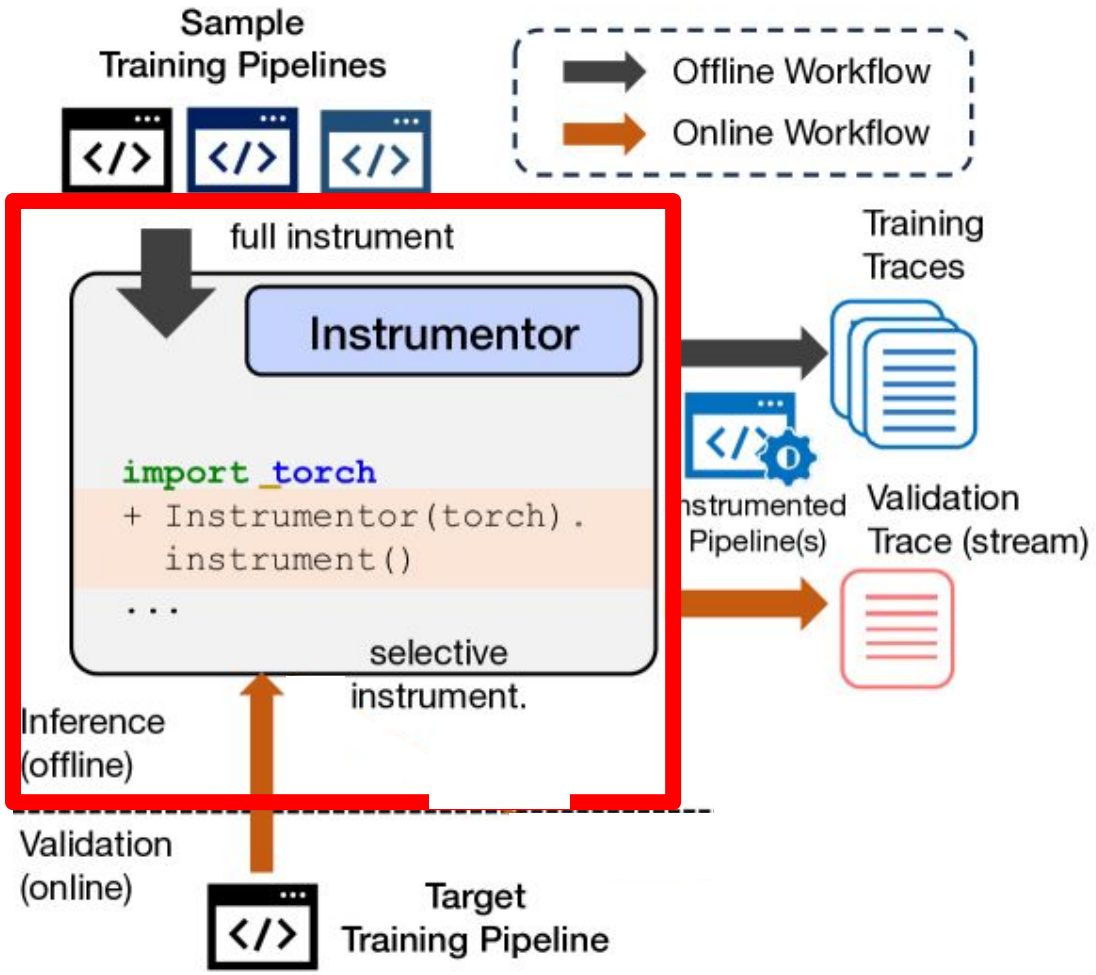
Current Solutions

Solution	Drawbacks
Manually create and write training invariants	Impractical for developers to manually create invariants for every training implementation
Spike/Trend/Anomaly Detectors: monitor patterns (large spikes, inconsistent updates) in loss or accuracy	• patterns in loss do not necessarily indicate bugs • No information for debugging
PyTea/NeuRI: specifies constraints on APIs, focusing mainly on tensor shape	• Can detect very limited root causes

Proposed Solution: TrainCheck



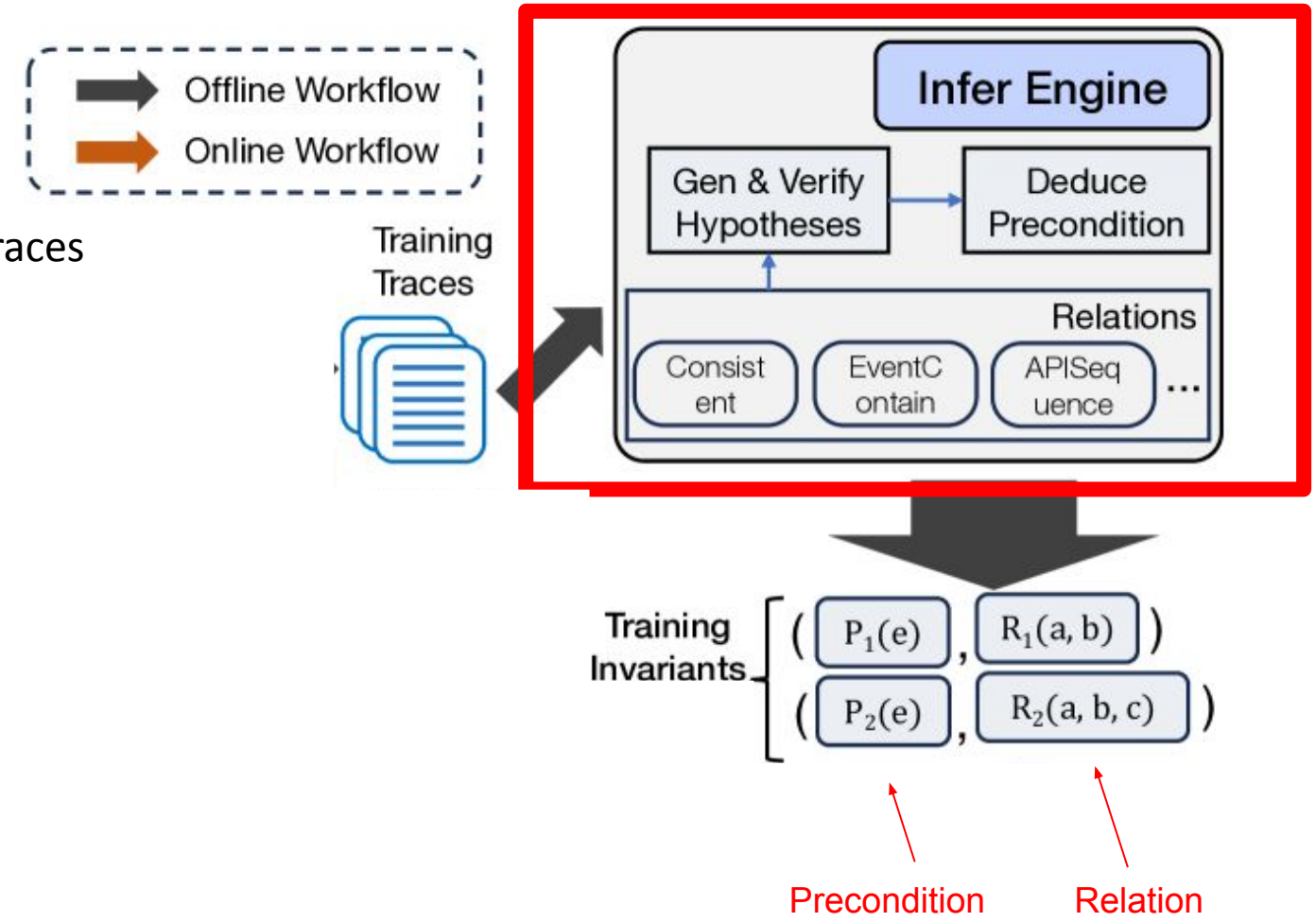
Instrumentor

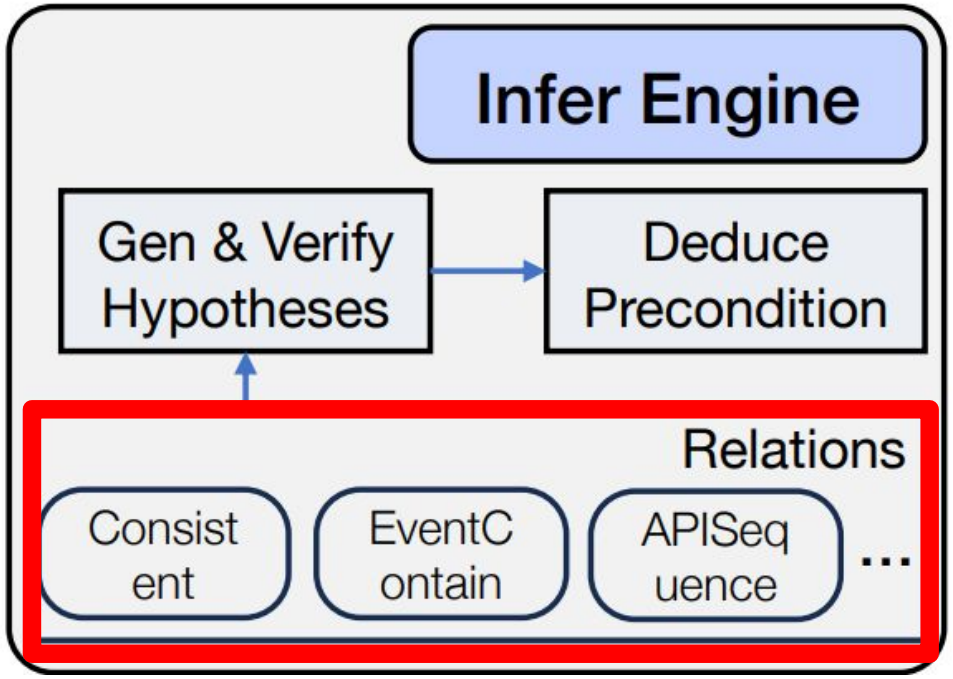


- Breaks training pipelines into traces required for training invariants
  - API Invocation Traces
  - Variable State Traces
  - Meta Variables

InferEngine

- Given a set of training traces from the Instrumentor, creates a set of training invariants.





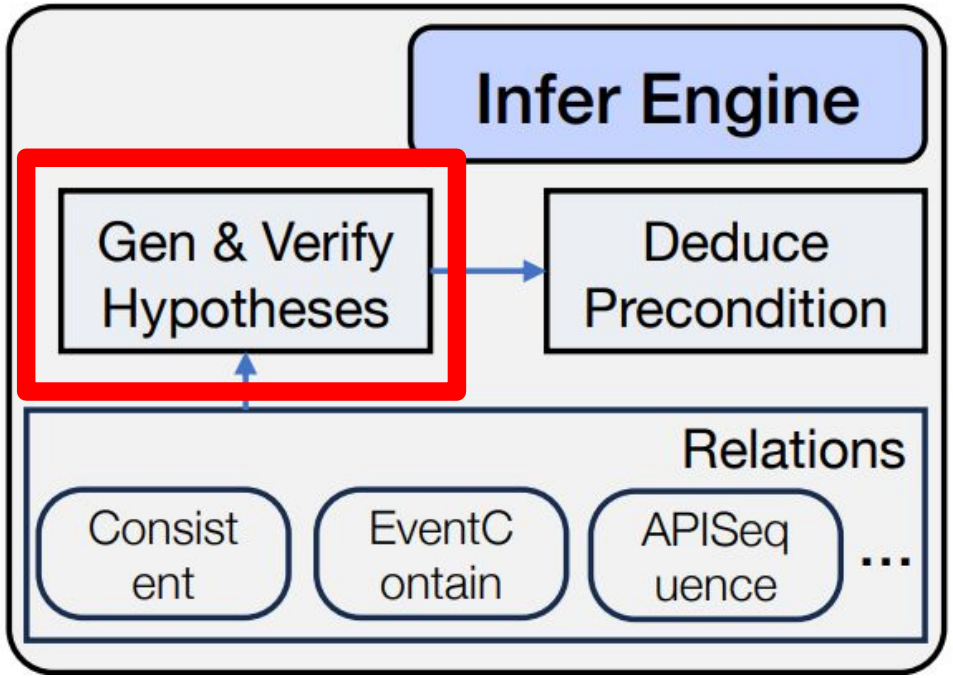
Trace snippet for torch.nn.Parameter

❶ {"name": "layernorm.weight", "type": "torch.nn.Parameter", "meta_vars": {"TP_RANK": 0, ...}, "attr": {"data": 411977, "is_cuda": true, "tensor_model_parallel": false, ...}}

❷ {"name": "layernorm.weight", "type": "torch.nn.Parameter", "meta_vars": {"TP_RANK": 1, ...}, "attr": {"data": 411977, "is_cuda": true, "tensor_model_parallel": false, ...}}

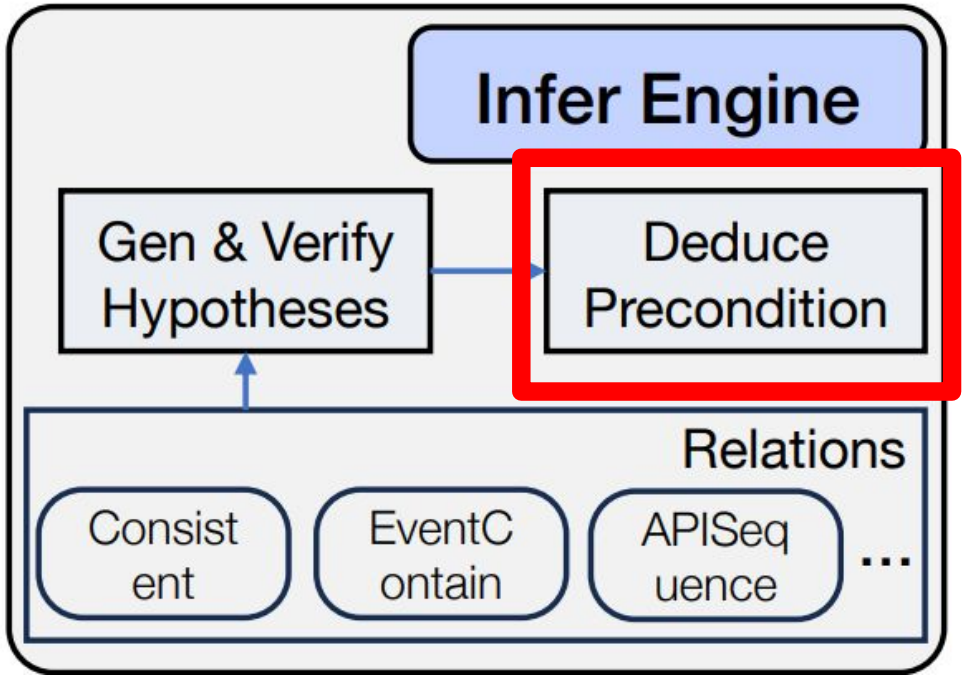
❸ {"name": "dense_h_to_4h.bias", "type": "torch.nn.Parameter", "meta_vars": {"TP_RANK": 1, ...}, "attr": {"data": 650462, "is_cuda": true, "tensor_model_parallel": true, ...}}

1. **Generate hypothesis**
   `CONSISTENT(torch.nn.Parameter.data, torch.nn.Parameter.data)`
2. **Validate hypothesis**
   Passing samples: (❶,❷) Failing samples: (❶,❸) (❷,❸)
3. **Deduce precondition**
   `UNEQUAL(meta_vars.TP_RANK) && CONSTANT(attr.tensor_model_parallel, false) && EQUAL(name)`



Algorithm 2: Hypothesis Generation for `Consistent`

Input: trace
Output: Generated hypotheses
variables ← trace.get_all_variables();
foreach $(var_1, var_2) \in Combinations(variables, 2)$ **do**
  foreach $(attr_1, attr_2) \in$
    $CartesianProduct(var_1.attrs, var_2.attrs)$ **do**
      if $exists\_value\_match(attr_1.states, attr_2.states)$
        hypo ←
          NEW_HYPO(*relation* ← ConsistentRelation,
            entities ← [VarDesc($var_1.type, attr_1.name$),
            VarDesc($var_2.type, attr_2.name$)]);
        yield hypo;



Trace snippet for torch.nn.Parameter

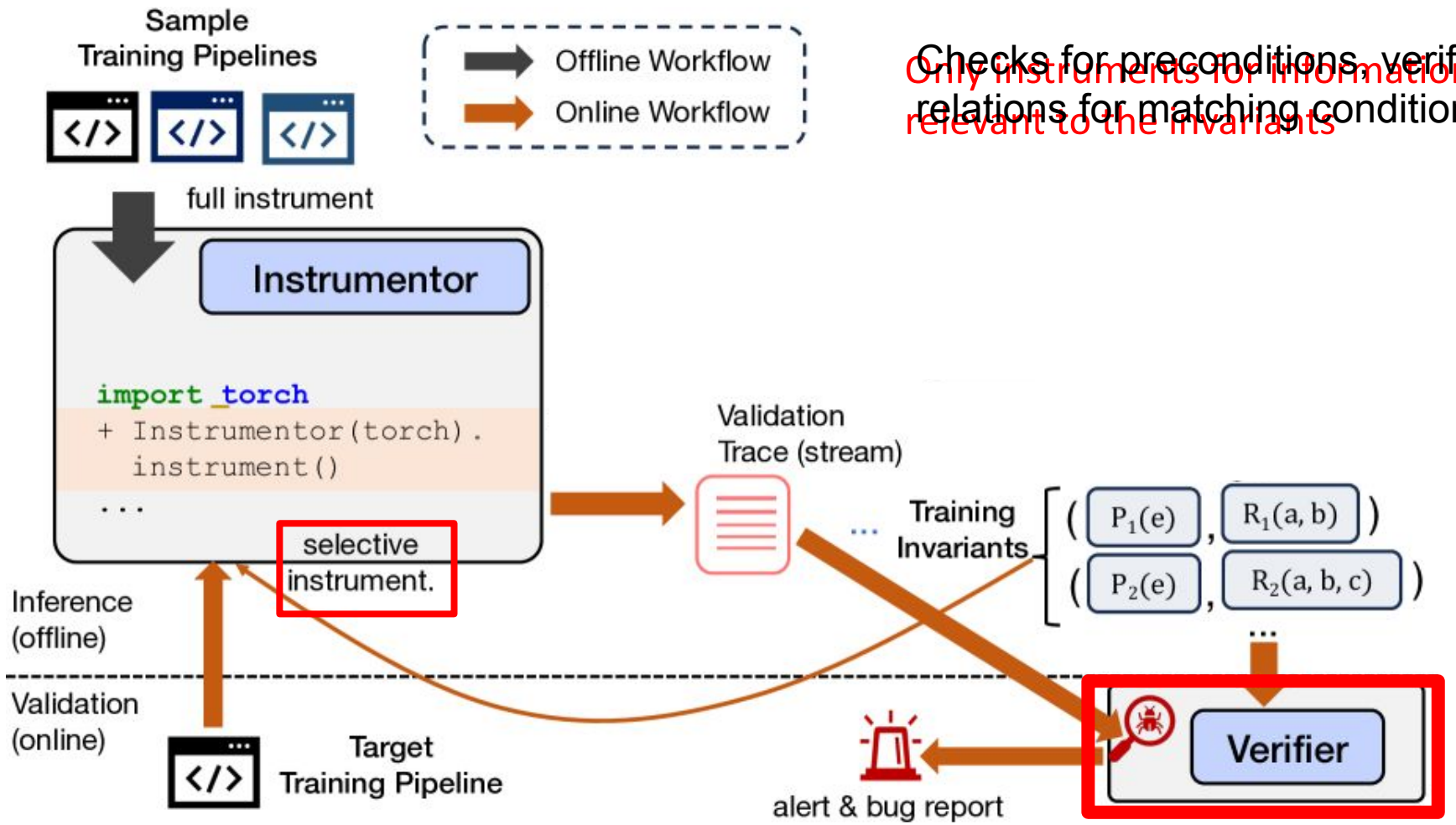
❶ {"name": "layernorm.weight", "type": "torch.nn.Parameter", "meta_vars": {"TP_RANK": 0, ...}, "attr": {"data": 411977, "is_cuda": true, "tensor_model_parallel": false, ...}}

❷ {"name": "layernorm.weight", "type": "torch.nn.Parameter", "meta_vars": {"TP_RANK": 1, ...}, "attr": {"data": 411977, "is_cuda": true, "tensor_model_parallel": false, ...}}

❸ {"name": "dense_h_to_4h.bias", "type": "torch.nn.Parameter", "meta_vars": {"TP_RANK": 1, ...}, "attr": {"data": 650462, "is_cuda": true, "tensor_model_parallel": true, ...}}

1. **Generate hypothesis**
   `CONSISTENT(torch.nn.Parameter.data, torch.nn.Parameter.data)`
2. **Validate hypothesis**
   Passing samples: (❶,❷) Failing samples: (❶,❸) (❷,❸)
3. **Deduce precondition**
   `UNEQUAL(meta_vars.TP_RANK)` `&&`
   `CONSTANT(attr.tensor_model_parallel, false)` `&&` `EQUAL(name)`

Verifier



Checks for preconditions, verifies relations for matching conditions

Only instruments for information relevant to the invariants

Evaluation: 18/20 Bugs Caught

- Evaluated on **20 test pipelines** with real-world bugs
- TrainCheck **caught 18** of the bugs and found **6 more unknown ones**
- 77% of the 18 caught after inferring invariants only from GCN

Bug Id	Synopsis
AC-2665	Initializing the optimizer prior to wrapping the model with DDP causes training to not progress.
DS-6770	A mismatch between the model and the parameters held by the optimizer causes a KeyError during initialization.
DS-5489	Freezing parameters prior to initializing DeepSpeed causes incomplete model checkpoints.
DS-6714	Using heterogeneous MoE architecture with pipeline parallelism causes inconsistent usage of communication primitives, leading to training stuck.
DS-6772	DeepSpeed initialization silently overwrites "id" attributes on models, causing wrong model-GPU placement.
DS-6089	The program is stuck on communication due to consistent "capacity" value across workers.

Table 3: Six newly reported bugs that lead to silent errors, detected and diagnosed with the help of TRAINCHECK. AC: Accelerate. DS: DeepSpeed. Numbers refer to GitHub issue IDs.

Evaluation Methodology

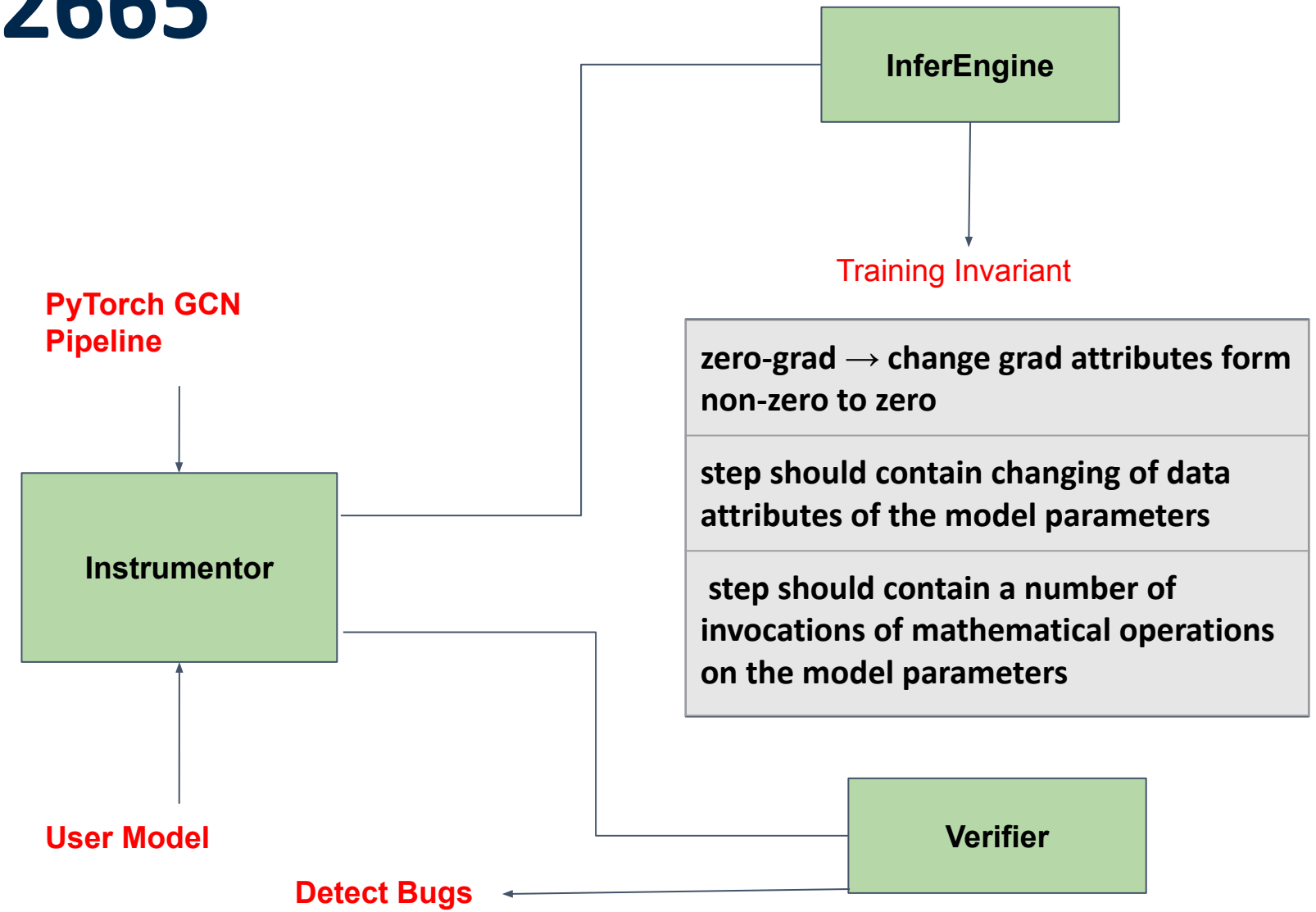
- Compared against current practices and state-of-the-art research:
  - Spike detector - monitors numerical instability in loss, accuracy
  - Trend detector - checks if loss/accuracy trends are as expected
  - Anomaly Detection - more algorithms to check loss/accuracy anomalies
  - PyTea and NeuRI - specify shaping constraints inputs/outputs of tensors
- Signal based detectors only detect **2 out of 20** example silent errors
- PyTea and NeuRI detect **1** more

Case Study: AC-2665

Issue: User adapted their model to use DDP (Distributed Data Parallel), but the model **stopped learning**

Step 1: Use standard PyTorch code to detect bugs

Step 2: Run under the PyTorch Instrumentor, infer invariants



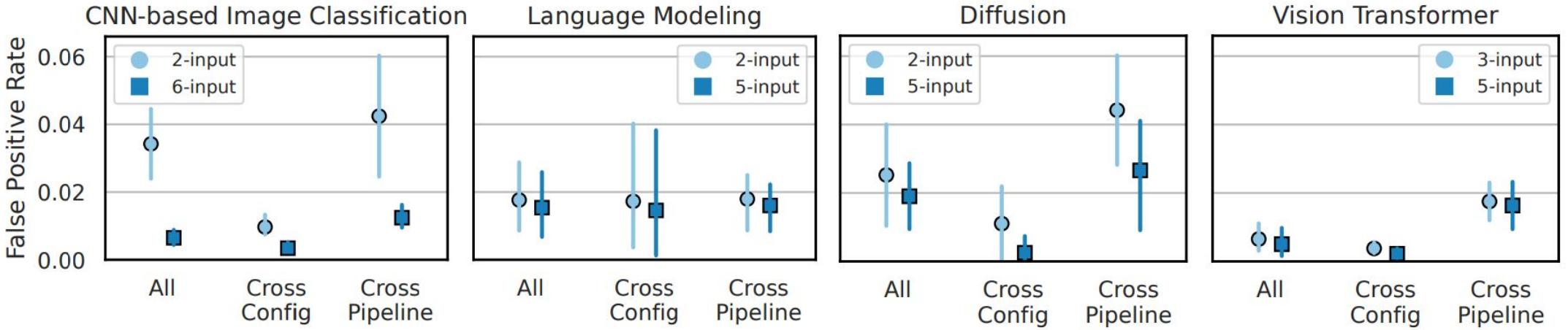
What was found

- Bug report showed that the optimizer was **not performing updates** to model parameters, and **no gradients were being computed**

- Further digging revealed that DDP flattens parameters and **creates a new model with those parameters,** but the user had initialized the optimizer with the **original parameters**

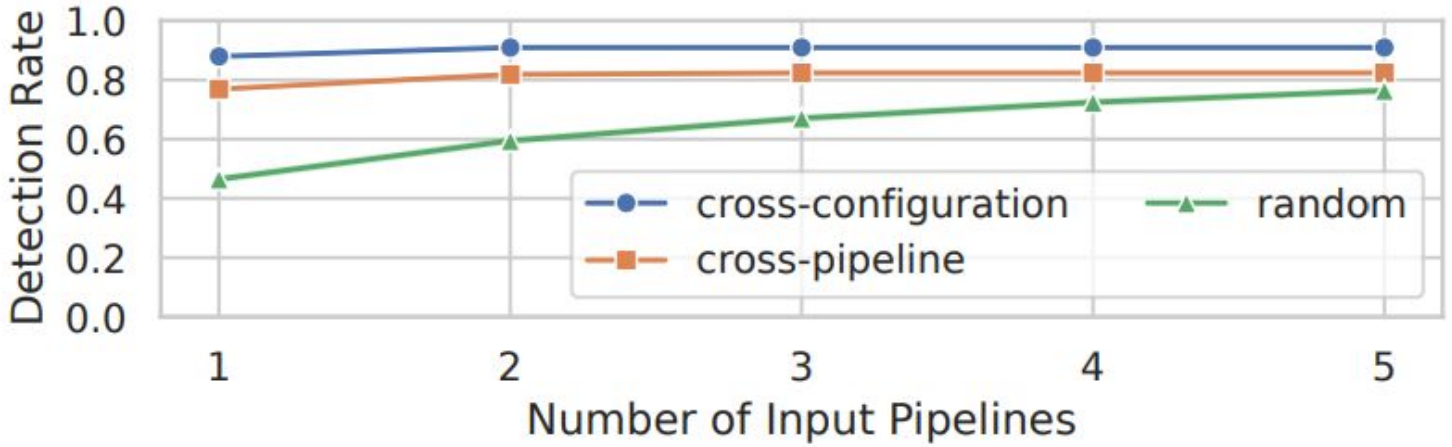
Evaluation: False Positives

- Tested on 63 training pipelines without known bugs
- Grouped into 4 classes by training type
- < 2% false positive rate due to pruned preconditions



Transferability of Invariants

- Transferability: Using learned invariants across different pipelines
- Tested on 3 classes of pipelines (relative to input pipeline):
  - Cross-configuration
  - Cross-pipeline
  - Random
- Detection rate improves with number of input pipelines



Runtime Overhead

- < 2% Runtime inference overhead on larger pipelines
- Up to 1.6x slowdown on smaller toy workloads like GCN (PyTorch's Autocast)
- Overhead mainly due to trace serialization to JSON

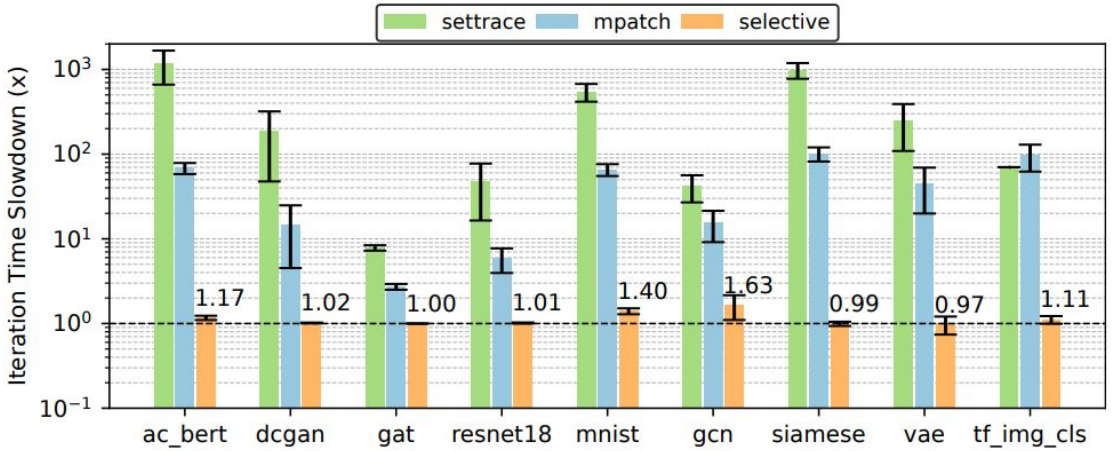
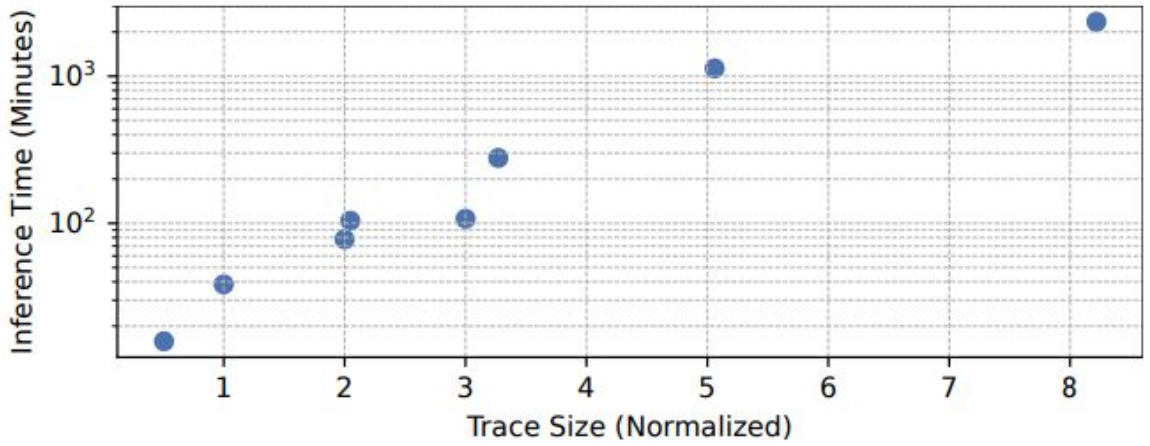


Figure 10: Overhead of different instrumentation techniques.

Inference Inefficiency

- Inference of invariants is roughly quadratic w.r.t input trace size
- Infers invariants from traces of 8 programs in ~38 hours
- Single threaded implementation
- Offline, so acceptable



TrainCheck Limitations

- Interferes with JIT compilation tools like torch.compile
- Limited to Python
- Cannot analyze Tensors in hash form

TrainCheck Summary

- Silent errors can’t be detected easily from loss/accuracy
- TrainCheck uses offline **learned invariants** to check training pipelines **at runtime**
- Provides **hints** for finding the source of bugs
- Has a **high detection rate** and **low runtime overhead**

Q&A